

Graduation Project

Devotopia

Verified Skill Roadmaps

An AI-Driven Platform for Personalised Learning Paths and

Evidence-Based Technical Hiring

Adaptive Assessment · Curriculum Generation · Verified Talent Pipeline

Supervised by:

< Project Supervisor >

By:

Ali Maher — Project Leader

Mohamed El-Saeed

Nada Nasr

Marina George

Intake < no. > < Month, Year >

[This page is left blank intentionally]

Acknowledgment

The completion of this project is not the result of our efforts alone. It rests on the guidance, patience and encouragement of many people, and we are grateful to all of them.

Our thanks go first to our project supervisor, whose direction shaped this work from an idea into a system. The questions asked of us at each review were consistently the questions that mattered, and several of the decisions documented in this report — particularly the choice to keep assessment logic deterministic rather than delegating it to a language model — were sharpened considerably by that scrutiny.

We extend our gratitude to the Information Technology Institute for providing the environment, the training and the opportunity to work on a project of this scope, and to the instructors of the Open Source track whose teaching over the preceding months laid the technical foundation this project is built on.

We also thank the students and engineers who gave their time to answer our questions about how they learn and how they hire. Their answers changed what we built.

Finally, we thank our families and friends for their support throughout. This project demanded long hours across several months, and that time was made available to us by the patience of the people around us.

Abstract

The technical hiring market suffers from an information problem that harms both sides of it. On one side, a graduate deciding to become a software developer must assemble their own curriculum from scattered and unordered material, with no reliable signal telling them when a topic is genuinely finished. On the other, an employer screening junior candidates receives large numbers of curricula vitae that list identical, entirely self-declared skills, with no evidence behind any of them. The result is misdirected study for the learner and an expensive, low-signal screening process for the employer.

This project addresses that problem by producing a platform in which learning and verification are the same activity. A learner states a target job role; the system generates a dependency-ordered curriculum for that role using large language models, and gates every module in it behind an adaptive assessment. The assessment adjusts question difficulty in response to the learner's recent answers, scores the result by the difficulty actually faced rather than by raw question count, and treats failure as a signal to rewrite the curriculum rather than merely as a mark to record.

The outcome of that process is a verified skill record. Because each verified skill carries the difficulty at which it was demonstrated, it can be searched and ranked by employers, which turns the learner's study effort directly into hiring signal. The platform closes the loop by comparing verified skills against real job requirements and allowing any identified gap to be injected back into the learner's roadmap.

The system is implemented as a TypeScript monorepo comprising a Next.js client and a NestJS application programming interface, backed by MongoDB and a Qdrant vector store, with forty feature modules and one hundred and eighteen mapped routes. A deliberate architectural constraint runs through the implementation: the language model generates content, but never determines control flow. Every decision that affects a learner's outcome is deterministic, testable application code. This constraint is what makes the assessment rules unit-testable, and it was validated in practice when all three configured model providers failed simultaneously and the platform continued to operate.

Preface

This documentation was prepared for the graduation project of the Open Source track at the Information Technology Institute, Alexandria branch.

The report is written to be read in order. Chapters 1 and 2 establish the problem and survey what already exists to address it. Chapter 3 documents the requirements engineering process, following the four phases of elicitation, classification, prioritisation and specification for both functional and non-functional requirements. Chapter 4, the longest, covers design and implementation, and gives particular attention to the two subsystems where the substantive engineering decisions were made: the adaptive assessment engine and the multi-provider model orchestration layer. Chapters 5 through 8 cover deployment, conclusions, planned future work and references.

Where a design decision had viable alternatives, this report states the alternatives and the reason for the choice rather than presenting the outcome as though it were inevitable. Where a measurement is quoted, it was taken from the running system and the method is stated alongside it.

Table of Contents

Acknowledgment	3
Abstract	4
Preface	4
1. Introduction	8
1.1 General	8
1.2 Motivation	8
1.3 Scope	9
1.3.1 In scope	9
1.3.2 Out of scope	9
2. Motivation & Background	10
2.1 Related Work	10
2.1.1 Industry Work	10
2.1.2 Survey Work	11
2.1.3 Adaptive Assessment and AI in Education	12
2.2 Data Sources	13
2.3 Market Needs	13
2.3.1 The learner's market	13
2.3.2 The employer's market	13
2.3.3 Why both sides are required	13
2.4 Project Needs	14
3. Requirements Elicitation & Analysis	15
3.1 Functional Requirements Elicitation & Analysis	15
3.1.1 Requirements discovery	15
3.1.2 Requirements classification and organisation	15
3.1.3 Prioritisation and negotiation	15
3.1.4 Requirements specification	16
3.2 Non-Functional Requirements Elicitation & Analysis	21
3.2.1 Requirements discovery	21
3.2.2 Requirements classification and organisation	21
3.2.3 Prioritisation and negotiation	21
3.2.4 Requirements specification	21
4. Design and Implementation	23
4.1 Architectural Design	23
4.1.1 System architecture: client–server	23

4.1.2 Layer responsibilities	24
4.1.3 Application architecture: modular monorepo	24
4.2 Frontend Development	25
4.2.1 Framework selection	25
4.2.2 Packages and tools	25
4.2.3 Interface design principles	25
4.2.4 User Interface Walkthrough	26
4.3 Backend Technology	51
4.3.1 The need for a backend framework	51
4.3.2 Framework selection: NestJS	51
4.3.3 Database selection: MongoDB	51
4.4 Backend Development	52
4.4.1 Database model	52
4.4.2 Security aspects	55
4.5 The Adaptive Assessment Engine	56
4.5.1 Overview	56
4.5.2 Session creation	56
4.5.3 Difficulty selection	56
4.5.4 Weighted scoring	56
4.5.5 Outcome and unlocking	57
4.5.6 The remedial loop	57
4.5.7 Known limitations	58
4.6 AI Curriculum Generation and Provider Orchestration	59
4.6.1 What is generated	59
4.6.2 The provider chain	59
4.6.3 Degradation observed in practice	59
4.6.4 The boundary between model and logic	59
4.6.5 Retrieval	60
4.7 External Services Integration	61
4.8 Performance Engineering	62
4.8.1 Measurement method	62
4.8.2 Findings and remedies	62
4.8.3 Measured outcome	62
4.9 Testing and Quality Assurance	63
5. Deployment	64
5.1 Deployment options	64
5.2 Reference deployment	64

5.3 Configuration	64
5.4 Operational notes	64
6. Conclusion	66
6.1 What was achieved	66
6.2 What was learned	66
6.3 Limitations	66
7. Future Plans	68
7.1 Assessment quality	68
7.2 Retrieval and model usage	68
7.3 Product	68
7.4 Engineering	68
8. Citations and References	69
8.1 Frameworks and libraries	69
8.2 Model providers	69
8.3 Methods and standards	69
8.4 Platforms examined	69

1. Introduction

1.1 General

Becoming employable as a software developer has never been a matter of information scarcity. The material required to learn any part of the discipline is freely available, in greater volume and higher quality than at any previous point. The difficulty is of a different kind: the material is unordered, its prerequisites are implicit, its quality is uneven, and nothing in it tells a learner when they have understood a topic well enough to move on.

A learner therefore faces three distinct problems at once. The first is sequencing — knowing what must be learned before what. The second is sufficiency — knowing when a topic is finished. The third is evidence — being able to demonstrate to somebody else that the learning happened, in a form that person will accept.

The conventional answer to the third problem is the completion certificate, and it is a weak answer. A certificate attests that a learner reached the end of a course. It does not attest that they can do anything, because the assessment behind it is typically identical for every learner and can be retaken until it passes. Employers understand this, which is why such certificates carry little weight in screening, which in turn means the learner's effort produces no hiring signal.

The same information problem appears in a different form on the employer's side. A recruiter screening junior developers receives applications that are largely indistinguishable: the same technologies listed, in the same order, with no way to tell a candidate who has built something from one who has watched a tutorial. Screening therefore falls back on weak proxies — the university attended, the formatting of the document, the confidence of its phrasing — none of which correlate reliably with ability. Capable candidates are rejected, and the cost of finding the remainder is high.

These are not two problems. They are one problem observed from two directions: there is no trustworthy, portable signal of what a junior developer can actually do.

1.2 Motivation

The motivation for this project came from direct observation within our own cohort. Among students preparing for their first developer role, the same three failures recurred with enough regularity that they appeared structural rather than individual.

A) The unordered curriculum

Learners assembled study plans from whatever material surfaced first, and consequently attempted topics whose prerequisites they had not met. The typical outcome was not a clean failure but a slow one: material that should have taken days took weeks, and the learner attributed the difficulty to themselves rather than to the ordering.

B) The absence of a stopping condition

Without an external check, a learner deciding whether they "know React" is answering a question they are not equipped to answer. Self-assessment in a partially-learned domain is unreliable in a specific direction — the learner does not know what they have not yet encountered. Learners consequently either moved on too early or remained on a topic long after the returns had diminished.

C) The wall

The failure mode that motivated the central feature of this project is the one that follows a failed attempt. A learner who does not understand a topic returns to the same material and reads it again. If the material was the reason they did not understand it, repeating it does not help. They stall, and the stall is frequently where the learner abandons the path.

Existing platforms record that failure and offer a retake. None of them respond to it by changing what the learner is asked to study. That gap — between detecting a failure and doing something useful about it — is the specific

opportunity this project was built to address.

1.3 Scope

1.3.1 In scope

Area	Included capabilities
Adaptive learning	Curriculum generation from a target role; adaptive assessment; remedial module injection; prerequisite-based unlocking; automatic track certification.
Talent profile	Résumé upload and structured parsing; AI enhancement of experience statements; verified skill passport; shield badges; shareable public portfolio.
Hiring	Job listings; skill-gap analysis against verified skills; one-action gap closing into the roadmap; ranked candidate pipeline for employers.
Engagement	Daily streaks; achievement system; study calendar; community discussion spaces; mentor directory, booking and ratings; in-app messaging and notifications.
Support systems	AI study assistant grounded in retrieved material; semantic search; sandboxed coding challenges; mock interview; generated audio summaries.
Platform	Bilingual Arabic and English with right-to-left layout; light and dark themes; installable progressive web application; subscription billing; administrative moderation.

1.3.2 Out of scope

The following were considered and deliberately excluded. Each exclusion is a decision with a reason, not an omission.

Excluded	Reason for exclusion
Live proctoring and identity verification	Requires video capture, storage of biometric data, and a consent and privacy regime disproportionate to a graduation project. The credential is positioned as a screening signal, not as a certification of record.
Native mobile applications	The progressive web application delivers installability and offline shell on both major platforms without maintaining two additional codebases within the project period.
Automated job scraping from external boards	Legally and contractually constrained, and unstable to maintain. Job records are seeded and employer-posted instead.
Payroll, contracts and post-hire processes	The product ends at the point of match. Extending into human-resources workflow would broaden the domain beyond what the team could implement to a defensible standard.
Content authoring by instructors	Curriculum content is generated rather than authored. An authoring tool would make the platform a learning-management system, which is a different product.

2. Motivation & Background

2.1 Related Work

2.1.1 Industry Work

Before designing the system we examined the products already serving this space, in two groups: global platforms with substantial market share, and regional platforms serving Arabic-speaking learners. The purpose was to establish what already exists, where it falls short, and whether the gap identified in section 1.2 is genuinely unaddressed.

2.1.1.1 Features comparison

Capability	Coursera	LinkedIn Learning	roadmap.sh	Almentor / Udacity Arabia	Devotopia
Generated personal curriculum	No	No	No	No	Yes
Prerequisite ordering enforced	Partial	No	Shown only	Fixed syllabus	Yes
Assessment adapts to the learner	No	No	None	No	Yes
Score weighted by difficulty faced	No	No	None	No	Yes
Failure changes what is studied next	No	No	None	No	Yes
Verified skill record for employers	Certificate	Badge	None	Certificate	Exam-backed
Employer-facing candidate pipeline	Separate product	Jobs marketplace	None	No	Integrated
Gap analysis against real postings	No	Partial	No	No	Yes
Arabic interface with RTL layout	Partial	Partial	No	Yes	Yes

Table 2.1 — Capability comparison against existing platforms.

2.1.1.2 Competitors' weaknesses

Examining the comparison above, four weaknesses recur across the platforms studied.

- **Verification by attendance** — the dominant model issues a credential for finishing a course. Because the final assessment is identical for every learner and retakeable without limit, the credential attests to persistence rather than to ability. Employers discount it accordingly, and the learner receives little return on the effort.
- **Assessment decoupled from learning** — where a skill test exists, it is typically a standalone quiz unconnected to any learning path. LinkedIn's skill assessments are a fixed set of questions, and a failed attempt is simply hidden from the profile — the learner receives no diagnostic information and the path they were following does not change.
- **Static, identical paths** — community roadmaps such as roadmap.sh solve the sequencing problem well, and were a genuine influence on this project. But the same roadmap is published to every visitor regardless of what they already know, and progress is self-ticked, so the sequencing benefit comes with no verification at all.
- **No feedback from failure** — across every platform examined, a failed assessment produces a retake offer. None regenerate or reorder the learner's material in response. The system detects the failure and then does nothing with the information.

2.1.1.3 International platforms

We also examined platforms outside the direct competitive set for techniques worth adopting.

Platform	Technique of interest	Applicability here
Duolingo	Spaced repetition and a difficulty model that reacts to recent performance; streaks as a retention mechanism.	The difficulty-reaction principle informed our assessment design. Streaks were adopted directly.
Khan Academy	Mastery-based progression — a topic is not complete until demonstrated, and prerequisites gate what is available.	Directly analogous to our module unlocking; validated the prerequisite-graph approach.
Pluralsight Skill IQ	Adaptive testing that estimates a level in relatively few questions.	The closest existing analogue to our engine. Confirmed that a short adaptive test can produce a usable signal.
HackerRank / Codility	Employer-facing verified assessment as the core product.	Validated employer appetite for verified signal, but these test in isolation from any learning path.

Table 2.2 — Techniques examined outside the direct competitive set.

2.1.1.4 Solutions offered by our application

Against the four weaknesses identified above, the platform offers four corresponding mechanisms.

- Verification by demonstrated performance rather than attendance. A skill is recorded only when the learner has answered questions about it and passed a weighted threshold, and the record carries the difficulty at which it was demonstrated.
- Assessment bound to the curriculum. Every exam belongs to a specific module in a specific roadmap, so its result can act on the learner's path rather than on a detached profile badge.
- Curriculum generated per learner. The roadmap is produced from the learner's stated target role, current skills and experience, so a learner who already knows a topic is not asked to repeat it.
- Failure as an input. Falling below the pass threshold causes the missed topics to be collected and a shorter remedial module to be generated from them and inserted ahead of the failed module.

2.1.2 Survey Work

To confirm that the problems described in section 1.2 are general rather than particular to our own experience, a questionnaire was distributed to learners preparing for technical roles, supplemented by semi-structured interviews with people responsible for junior hiring.

A) Method

Parameter	Value
Instrument	Online questionnaire (learners) and semi-structured interviews (employers)
Learner population	Final-year and recently graduated computer science and information systems students
Employer population	Engineers and managers responsible for screening junior developers
Learner sample size	< n = ? >
Employer sample size	< n = ? >
Collection period	< start date > to < end date >

B) Findings

#	Question	Result	Design decision it drove
1	How do you currently decide what to learn next?	< % >	Justifies generated, prerequisite-ordered roadmaps over a static catalogue.
2	Have you abandoned a learning path before finishing it?	< % >	Justifies the remedial loop rather than a repeated module.

#	Question	Result	Design decision it drove
3	What made you stop?	< % >	Confirms whether "the wall" described in 1.2C is the dominant cause.
4	Do you believe a certificate of completion helps you get interviews?	< % >	Establishes the weakness of attendance-based credentials from the learner side.
5	Would a verified skill record change that?	< % >	Justifies the skill passport and public profile.
6	(Employers) What do you trust most when screening a junior CV?	< % >	Establishes what signal the pipeline must expose to be useful.
7	(Employers) How long does screening one junior role take?	< hours >	Quantifies the cost the pipeline is intended to reduce.
8	Would you pay for AI-generated roadmaps and assessments?	< % >	Sets the consumer price point.

Table 2.3 — Survey instrument and the design decisions each question informs.

This section requires real data before submission

Every value in the two tables above is a placeholder. Replace them with the actual counts and percentages from your questionnaire, and add the corresponding charts — a bar chart for questions 1 and 6, a pie chart for question 2.

State the sample size honestly. A stated $n = 40$ with real numbers is defensible under questioning; impressive percentages that cannot be sourced are not, and assessors do ask how the data was collected.

If you have spoken to a real company or a real learner cohort, name them here. A single verified customer contact carries more evidential weight than any other item in this chapter.

2.1.3 Adaptive Assessment and AI in Education

2.1.3.1 Applications of AI in learning platforms

Language models are applied to education in several distinct roles, and it is worth separating them because they carry very different risk profiles.

- **Content generation** — producing explanations, examples, exercises and assessment items. The failure mode is inaccuracy in the generated material, which is visible to the learner and correctable.
- **Personalisation** — selecting or ordering material for a specific learner. The failure mode is a poor path, which is harder for the learner to detect because they cannot see the alternative.
- **Assessment and grading** — evaluating a learner's response. The failure mode is a wrong judgement about a person, which is both invisible to the learner and consequential.
- **Conversational tutoring** — answering questions in dialogue. The failure mode is confident inaccuracy, mitigated by grounding responses in retrieved source material.

This project uses models in the first and fourth roles, and deliberately restricts the second and third. Curriculum content and assessment items are generated; the decisions that determine a learner's outcome are not. Section 4.5 sets out that boundary in detail and the reasoning behind it.

2.1.3.2 Types of adaptive assessment

Three approaches to adaptive assessment were considered.

Approach	How it works	Strengths	Why it was or was not adopted
Item Response Theory (IRT) / Computerised Adaptive Testing	Each question carries calibrated difficulty and discrimination parameters; the learner's ability is estimated continuously and the next item is chosen to maximise information.	Statistically principled; scores comparable across different question sets; efficient in question count.	Not adopted. Calibration requires a large body of prior response data per item, which a new platform does not have. Recorded as future work in chapter 7.

Approach	How it works	Strengths	Why it was or was not adopted
Rule-based streak adaptation	Difficulty moves up or down according to a defined pattern in the learner's recent answers.	Requires no prior calibration data; deterministic and therefore unit-testable; transparent enough to explain to a learner.	Adopted. It is the only approach of the three that is both defensible without historical data and testable as ordinary code.
Model-judged assessment	A language model both poses questions and judges whether the learner has demonstrated understanding.	Handles free-text answers; requires no fixed item bank.	Rejected. It places a consequential judgement about a person inside a non-deterministic component that cannot be unit-tested, and makes an outcome dependent on provider availability.

Table 2.4 — Adaptive assessment approaches considered.

2.2 Data Sources

The platform draws on five categories of data, each with a different provenance and a different reliability requirement.

Source	Data obtained	Use	Reliability treatment
Learner-supplied	Target role, current skills, education, experience, uploaded résumé.	Input to curriculum generation and to the profile.	Trusted as self-declaration only; never treated as verified skill.
Generated	Roadmap modules, assessment items, remedial modules, enhanced CV statements.	Curriculum and assessment content.	Shape-checked before acceptance; a malformed reply is discarded and the next provider tried.
System-recorded	Answer history, scores, pass and fail records, streaks, per-topic statistics.	The verified skill record and all analytics.	Authoritative — written only by server-side code from real interactions.
Employer-supplied	Job postings, required skills, company profiles.	Matching and gap analysis.	Trusted for matching; not used to alter learner records.
External services	Salary insight data, speech transcription, media hosting.	Supplementary display and features.	Advisory only; failure degrades a feature without affecting core function.

The distinction between the first and third rows is the substance of the product. Self-declared skills enter the system freely and are never presented as verified. Only the system-recorded outcome of an assessment produces a verified skill, and that record is written exclusively by server-side code from an actual answer history.

2.3 Market Needs

2.3.1 The learner's market

The addressable learner population comprises computer science and information systems graduates, self-taught developers, and career switchers entering technology from other fields. Their common requirement is not more material — it is ordering, a stopping condition, and evidence.

2.3.2 The employer's market

Junior technical hiring is characterised by high applicant volume, low signal per applicant, and a screening cost borne by engineers whose time is expensive. An employer benefits from any mechanism that raises signal per applicant, provided the mechanism is credible. This is the condition the platform must satisfy: the verified record has to be trustworthy enough that a recruiter will act on it.

2.3.3 Why both sides are required

A platform serving only learners produces credentials nobody recognises. A platform serving only employers has no supply of verified candidates. The two sides are mutually dependent, which is a structural difficulty at launch and the

reason the go-to-market plan in section 2.4 begins with a single institution containing both.

2.4 Project Needs

For the system described to be viable, the following were required.

Need	Rationale
Generation that respects prerequisites	A roadmap that does not order its modules correctly solves nothing; ordering is the first of the three learner problems.
Assessment that cannot be gamed by repetition	If an exam can be retaken until it passes, the resulting record carries no information and the employer side collapses.
A response to failure other than repetition	This is the differentiating mechanism and the answer to the failure mode described in section 1.2C.
Deterministic outcome logic	A decision about a person must be explainable and testable. This constraint shaped the entire architecture.
Graceful degradation	A platform dependent on external model providers must remain usable when they are unavailable, or it cannot be relied upon at all.
Bilingual delivery	The target population is Arabic-speaking and reads technical material in English; both must be first-class.

3. Requirements Elicitation & Analysis

Requirements were developed through the four standard phases — discovery, classification and organisation, prioritisation and negotiation, and specification — applied separately to functional and non-functional requirements.

3.1 Functional Requirements Elicitation & Analysis

3.1.1 Requirements discovery

Four techniques were used, chosen so that their failure modes differ from one another.

Technique	Applied to	What it produced
Competitor analysis	The platforms in section 2.1.1	The baseline expectation set, and the four weaknesses the product must not reproduce.
Questionnaire	Learners preparing for technical roles	Relative importance of candidate features, and confirmation of the abandonment pattern.
Semi-structured interviews	Engineers and managers who screen junior candidates	What signal the employer side must expose to be acted upon.
Team domain experience	Our own preparation for the same job market	The specific failure mode of section 1.2C, which surveys tend to under-report because learners attribute the failure to themselves.

3.1.2 Requirements classification and organisation

The discovered requirements were organised into seven functional groups, each corresponding to a coherent area of the system.

Group	Scope of the group
G1 — Identity and access	Registration, authentication, session handling, roles, profile management.
G2 — Curriculum	Onboarding intake, roadmap generation, module states, prerequisite unlocking, gap injection.
G3 — Assessment	Exam sessions, adaptive difficulty, weighted scoring, pass and fail handling, remedial generation, certification.
G4 — Talent profile	Résumé upload and parsing, AI enhancement, verified skill passport, badges, public portfolio.
G5 — Hiring	Job postings, skill matching, gap analysis, candidate pipeline, employer search.
G6 — Engagement and support	Streaks, achievements, calendar, community, mentors, messaging, notifications, study assistant, coding practice.
G7 — Administration and billing	Moderation, reporting, audit logs, analytics, subscriptions and payment.

3.1.3 Prioritisation and negotiation

Requirements were prioritised using the Kano model, which classifies features by their effect on user satisfaction rather than by implementation cost. Features already filtered as indifferent were excluded, leaving three classes.

Basic — expected, dissatisfying if absent	Performance — satisfaction rises with quality	Excitement — differentiating
Registration and authentication	Curriculum quality and ordering	Remedial loop on failure
Roadmap generation	Assessment question quality	Difficulty-weighted verified skills
Module progression and unlocking	CV parsing accuracy	One-action skill-gap closing
Assessment with a pass mark	Job match relevance	Employer pipeline ranked by exam performance

Basic — expected, dissatisfying if absent	Performance — satisfaction rises with quality	Excitement — differentiating
Profile and CV storage	Interface responsiveness	AI study assistant grounded in retrieved material
Bilingual interface	Streaks and achievements	Sandboxed coding challenges and mock interview

Table 3.1 — Functional requirement prioritisation using the Kano model.

The basic class carries the highest implementation priority because its absence makes the product unusable rather than merely worse. The performance class was targeted next, since these attributes determine whether the product is judged good or poor. The excitement class contains the features that distinguish the product from those surveyed in chapter 2 — and notably, the remedial loop appears there, which is the reason it was protected from being cut when the schedule tightened.

Negotiated reductions

Three requirements were reduced in scope during prioritisation, and the reasoning is recorded here because the reductions are visible in the delivered system.

Requirement	Original intent	Delivered	Reason
Exam length	Ten to fifteen items per module for a stable difficulty estimate.	Five items.	Completion rate falls sharply with exam length. Accepted the reduced statistical resolution; recorded as a known limitation in section 4.5.7.
Job data	Live ingestion from external job boards.	Seeded and employer-posted records.	Terms-of-service constraints and maintenance instability. Deferred to chapter 7.
Assessment scope	Free-text and code answers judged by a model.	Multiple choice with deterministic grading.	A model-judged outcome cannot be unit-tested, and this is a decision about a person. See section 2.1.3.2.

3.1.4 Requirements specification

The specification below states each functional requirement with an identifier, the responsible actor, and its Kano class. Identifiers are referenced throughout chapter 4.

ID	Requirement	Group	Actor	Class
FR-01	The system shall allow registration and authentication by email and password, or by Google identity.	G1	All	Basic
FR-02	The system shall reject the administrator role at public registration.	G1	System	Basic
FR-03	The system shall issue a short-lived access token and a rotating refresh token bound to the device.	G1	System	Basic
FR-04	The system shall capture target role, education, experience and current skills during onboarding.	G2	Learner	Basic
FR-05	The system shall generate a roadmap of modules with hour estimates, topics, prerequisites and layout coordinates.	G2	Learner	Basic
FR-06	The system shall present the roadmap as a navigable tree showing each module's state.	G2	Learner	Basic
FR-07	The system shall permit an assessment to start only for a module that is not locked.	G3	Learner	Basic
FR-08	The system shall present five questions per assessment, each with a stated difficulty and a time limit.	G3	Learner	Basic
FR-09	The system shall raise difficulty after two consecutive correct answers and lower it after two consecutive incorrect answers.	G3	System	Excitement

ID	Requirement	Group	Actor	Class
FR-10	The system shall score each assessment by the difficulty faced, weighting easy at 1.0, medium at 1.5 and hard at 2.0.	G3	System	Excitement
FR-11	The system shall record a pass when the weighted score reaches 70 per cent.	G3	System	Basic
FR-12	The system shall unlock dependent modules on a pass, following the prerequisite graph.	G3	System	Basic
FR-13	The system shall maintain a running fail percentage per topic per learner.	G3	System	Excitement
FR-14	The system shall generate a remedial module from the missed topics and insert it ahead of the failed module once that percentage reaches 30 per cent.	G3	System	Excitement
FR-15	The system shall issue a track certificate when every module in a track is completed.	G3	System	Performance
FR-16	The system shall accept a résumé in PDF form and parse it into structured sections.	G4	Learner	Basic
FR-17	The system shall enhance experience statements on request while preserving the original.	G4	Learner	Performance
FR-18	The system shall present a skill passport of verified milestones with the score and date of each.	G4	Learner	Excitement
FR-19	The system shall expose a shareable public profile of verified skills.	G4	Learner	Performance
FR-20	The system shall compare verified skills against a posting's requirements and present the gap.	G5	Learner	Excitement
FR-21	The system shall inject missing requirements into the roadmap as new modules on request.	G5	Learner	Excitement
FR-22	The system shall allow an employer to publish a posting with required skills.	G5	Company	Basic
FR-23	The system shall present candidates ranked by verified exam performance and completion rate.	G5	Company	Excitement
FR-24	The system shall provide a study assistant whose answers are grounded in retrieved material.	G6	Learner	Excitement
FR-25	The system shall execute submitted code in an isolated sandbox and return the result.	G6	Learner	Excitement
FR-26	The system shall support mentor profiles, session booking and ratings.	G6	Learner / Mentor	Performance
FR-27	The system shall provide community discussion spaces with posting, commenting and reporting.	G6	Learner	Performance
FR-28	The system shall record daily activity, maintain streaks and award achievements.	G6	System	Performance
FR-29	The system shall present activity analytics over 7, 30 and 90-day windows.	G6	Learner	Performance
FR-30	The system shall resolve subscription prices server-side from the plan identifier.	G7	System	Basic
FR-31	The system shall verify the payment webhook signature before altering entitlement.	G7	System	Basic
FR-32	The system shall provide administrators with user management, moderation, reporting, audit logs and analytics.	G7	Admin	Basic
FR-33	The system shall support Arabic and English with right-to-left layout, and light and dark themes.	G1	All	Basic

Table 3.2 — Functional requirements specification.

A) Use case diagram

The diagram below shows the four human actors against the system boundary. Solid lines are associations; dashed arrows are «include» relationships, expressing the outcomes the system derives automatically once an assessment is submitted.

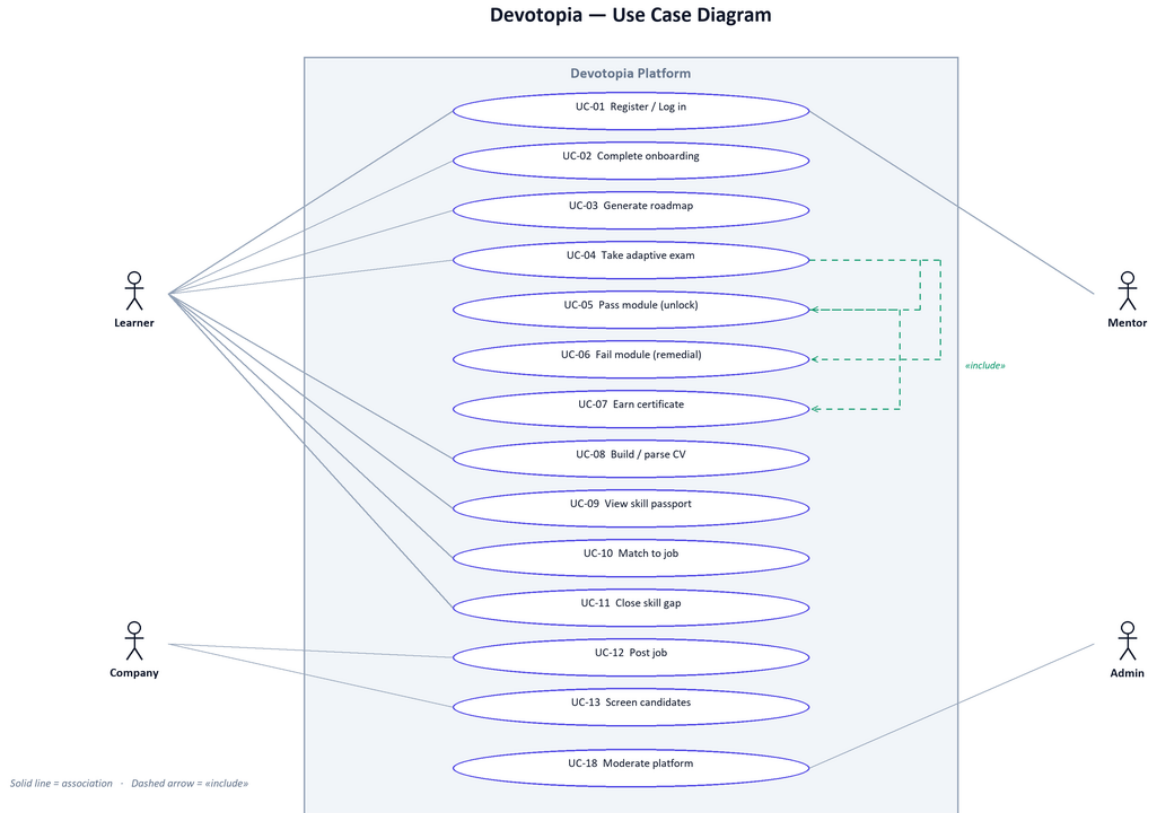


Figure 3.1 — Use case diagram.

B) Actors

Actor	Description	Principal requirements
Learner	Studies a roadmap, sits assessments, builds a verified profile.	FR-04 ... FR-21, FR-24 ... FR-29
Company / Recruiter	Publishes postings and screens verified candidates.	FR-22, FR-23, FR-30
Mentor	Advises learners and conducts mentorship sessions.	FR-26
Administrator	Operates and moderates the platform.	FR-32
System (autonomous)	Derives outcomes without human instruction.	FR-09 ... FR-15, FR-28, FR-31
External services	Model providers, payment processor, media and speech services.	Invoked server-side only.

C) Principal use cases

ID	Use case	Actor	Description
UC-01	Register / log in	All	Create an account or authenticate and receive a session.
UC-02	Complete onboarding	Learner	Supply target role, education, experience and skills.
UC-03	Generate roadmap	Learner	Receive an ordered module tree for the stated role.
UC-04	Take adaptive assessment	Learner	Answer five questions whose difficulty adapts.
UC-05	Pass module	System	Unlock dependents and record a verified skill.
UC-06	Fail module	System	Generate and insert a remedial module.
UC-07	Earn certificate	Learner	Receive a track credential on full completion.
UC-08	Build CV	Learner	Upload, parse and enhance a résumé.
UC-09	View skill passport	Learner	Review and share verified milestones.
UC-10	Match to job	Learner	Compare verified skills to a posting.
UC-11	Close skill gap	Learner	Inject missing requirements into the roadmap.
UC-12	Post job	Company	Publish a role with required skills.
UC-13	Screen candidates	Company	Browse candidates ranked by verified performance.
UC-14	Book mentorship	Learner	Request and rate a session with a mentor.
UC-15	Ask study assistant	Learner	Obtain a grounded answer about the current module.
UC-16	Solve coding challenge	Learner	Submit code for sandboxed execution.
UC-17	Subscribe	Learner / Company	Purchase a plan and receive entitlement.
UC-18	Moderate platform	Admin	Handle reports, users and audit logs.

D) Detailed use case: UC-04, Take adaptive assessment

The central use case is specified in full, since the behaviour of chapter 4 follows from it.

Field	Content
Identifier	UC-04
Actor	Learner (primary); System (secondary)
Precondition	The learner is authenticated and the target module is not locked.
Trigger	The learner starts an assessment on a module.
Main flow	1. The learner requests an assessment for a module. 2. The system creates a session and requests a question batch at medium difficulty. 3. The system presents the question, its difficulty, and a 30-second timer. 4. The learner selects an answer, or the timer expires. 5. The system records the answer with its difficulty and elapsed time. 6. The system determines the next difficulty from the two most recent answers. 7. Steps 3 to 6 repeat until five answers are recorded. 8. The system computes the weighted score and compares it to the pass threshold. 9. The system applies the pass or fail outcome and presents the result.
Alternative — pass	At step 9, where the score is at or above 70 per cent: the module is marked completed; dependent modules are unlocked; a verified skill is recorded; if every module in the track is now complete, a certificate is issued.
Alternative — fail	At step 9, where the score is below 70 per cent: the module is marked failed; the incorrectly answered questions and their topics are collected; the running fail percentage for the topic is recomputed; if it reaches 30 per cent, remedial generation is queued and the resulting module is inserted ahead of the failed one.
Exception — no answer	At step 4, where the timer expires: the answer is recorded as incorrect with the full time elapsed, and the flow continues at step 5.
Exception — generation unavailable	At step 2, where every model provider fails: the deterministic mock provider supplies the question batch and the assessment proceeds normally.
Postcondition	A completed session exists with five recorded answers, a weighted score, and an applied outcome.

Table 3.3 — Detailed specification of use case UC-04.

3.2 Non-Functional Requirements Elicitation & Analysis

3.2.1 Requirements discovery

Non-functional requirements were discovered from three sources: the constraints implied by the functional set, the operating environment of the target users, and the failure modes observed in the platforms studied in chapter 2.

- **Implied by function** — a verified credential implies integrity requirements; an employer-facing pipeline implies authorisation boundaries; a generated curriculum implies availability requirements on external providers.
- **Implied by environment** — the target population uses mid-range mobile devices on variable connections, which constrains payload size and imposes an offline expectation.
- **Implied by observed failure** — every platform studied depends on external services; the ones that fail visibly are those without a degradation path.

3.2.2 Requirements classification and organisation

Non-functional requirements were organised into seven categories following the standard quality attribute set.

Category	Concern
Performance	Perceived speed, payload size, main-thread responsiveness.
Security	Authentication, authorisation, data protection, abuse resistance.
Availability and reliability	Behaviour under partial failure of dependencies.
Compatibility	Devices, screen sizes, browsers, installability.
Usability and accessibility	Bilingual delivery, layout direction, visual stability, theme.
Scalability	Growth in users and in generated work.
Maintainability and testability	Type safety, test coverage, reproducible builds.

3.2.3 Prioritisation and negotiation

Non-functional requirements were prioritised by the consequence of failing them, which for this system is not uniform across categories.

Priority	Categories	Justification
Critical	Security; availability and reliability	A breach of authorisation destroys the credibility of the verified record, which is the product. Unavailability of a model provider must not stop the platform, because it is outside our control and was observed to occur.
High	Performance; usability and accessibility	Determines whether the product is judged usable on the devices the target population owns. Directly measurable, and therefore directly defensible.
Medium	Compatibility; maintainability and testability	Necessary for delivery and for the project to be assessable, but failure degrades rather than prevents use.
Deferred	Scalability beyond the pilot	Horizontal scaling was designed for but not load-tested. Stated honestly here rather than claimed.

3.2.4 Requirements specification

ID	Requirement	Acceptance criterion	Result
NFR-01	Public pages shall reach largest contentful paint quickly on a production build.	LCP below 2.5 s.	Achieved — 196 to 272 ms measured.
NFR-02	Pages shall not shift visually while loading.	Cumulative layout shift below 0.1.	Achieved — 0 on all measured routes.
NFR-03	No main-thread task shall block interaction.	Longest task below 50 ms.	Achieved — 0 ms on measured routes.

ID	Requirement	Acceptance criterion	Result
NFR-04	First-load payload shall suit a mid-range connection.	Heaviest route below 200 kB.	Achieved — dashboard reduced 316 kB to 161 kB.
NFR-05	Every route shall require authentication unless explicitly public.	A global guard denies by default.	Achieved — JwtAuthGuard registered globally.
NFR-06	Role and resource ownership shall be enforced server-side.	Cross-user access attempts are refused.	Achieved — verified by the smoke suite.
NFR-07	Passwords shall be stored irreversibly.	bcrypt at 12 rounds; no plaintext persisted.	Achieved.
NFR-08	Session tokens shall resist theft by page script.	Refresh token in an httpOnly cookie; access token in memory only.	Achieved.
NFR-09	Authentication endpoints shall resist credential stuffing.	Stricter rate limit on auth than elsewhere.	Achieved — 15/min versus 100/min.
NFR-10	Privilege escalation shall be impossible through registration.	The administrator role is rejected at validation.	Achieved.
NFR-11	The platform shall remain usable when model providers fail.	Core flows complete with every provider unavailable.	Achieved — observed under real simultaneous failure.
NFR-12	Optional infrastructure shall not be required to run.	Absence of the queue or vector store degrades gracefully.	Achieved — in-memory queue and retrieval mock.
NFR-13	The interface shall serve Arabic and English equally.	Full translation and layout mirroring on every screen.	Achieved.
NFR-14	The application shall be installable and work offline in shell form.	Manifest and service worker present.	Achieved.
NFR-15	The codebase shall be type-safe end to end.	Compilation with no type errors.	Achieved — shared Zod schemas across both apps.
NFR-16	Outcome-determining logic shall be unit-testable.	Assessment rules covered by tests.	Achieved — remedial-trigger specification.
NFR-17	Security properties shall be verified automatically.	An executable suite covering authorisation and abuse.	Achieved — 44 checks against a live API.
NFR-18	Generated work shall not block request handling.	Long generation dispatched to a queue.	Achieved — BullMQ with fallback.

Table 3.4 — Non-functional requirements specification and measured outcomes.

4. Design and Implementation

4.1 Architectural Design

4.1.1 System architecture: client–server

The system follows a client–server architecture with a clear separation between a presentation client and an application server that owns all business logic and all external integrations.

Alternatives considered

Option	Description	Why not adopted
Monolithic server-rendered application	Pages rendered entirely on the server, minimal client logic.	The roadmap canvas, live assessment timer and analytics are inherently interactive; server round-trips per interaction would be unusable.
Serverless functions	Each endpoint deployed independently.	Cold starts on generation endpoints would be visible to the learner, and the shared domain logic in assessment would be awkward to distribute.
Microservices	Assessment, curriculum, hiring and community as separate services.	Operational overhead disproportionate to a team of four within a fixed period; the domain does not yet have the independent scaling requirements that justify the split.
Client–server with a modular monolith	One deployable API organised into feature modules; a separate client.	Adopted. Retains module boundaries and a clean path to future extraction, without present-day distributed-systems cost.

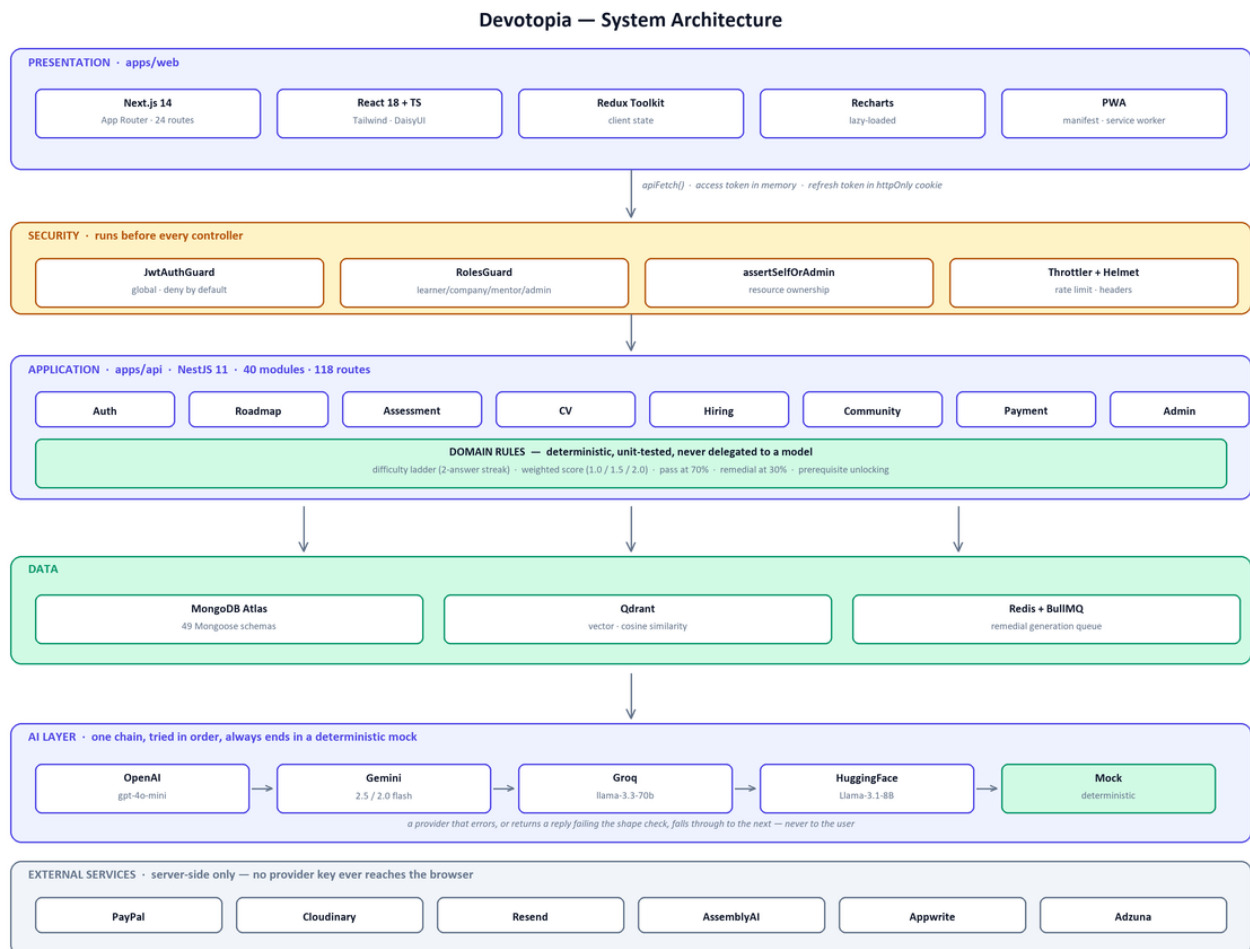


Figure 4.1 — System architecture.

4.1.2 Layer responsibilities

Layer	Responsibility	Implementation
Presentation	Rendering, interaction, client state, layout direction and theme.	Next.js App Router, 24 page routes.
Security	Authentication, role checks and ownership checks, executed before any controller.	Global guard, roles guard, ownership assertions.
Application	Request handling, validation, orchestration of domain operations.	NestJS, 40 feature modules, 118 routes.
Domain	The rules determining a learner's outcome.	Deterministic, unit-tested TypeScript.
Data	Persistence, retrieval, queued work.	MongoDB with 49 schemas, Qdrant, Redis.
Integration	All third-party communication.	Server-side only; no provider key reaches the browser.

4.1.3 Application architecture: modular monorepo

The repository is organised as a Turborepo monorepo with npm workspaces containing three packages.

Package	Contents	Purpose
apps/web	Next.js client — 24 page routes.	Presentation.
apps/api	NestJS server — 40 modules, 118 routes, 49 schemas.	Application, domain, data and integration.
packages/shared	Zod schemas and derived TypeScript types.	A single definition of the request and response contract, imported by both applications so a change to a payload shape becomes a compile error on both sides rather than a runtime failure on one.

The shared package is the mechanism that makes the client–server split safe. Without it, the contract exists twice — once in each codebase — and the two definitions drift. With it, there is one definition and both sides are checked against it at compile time.

4.2 Frontend Development

4.2.1 Framework selection

Framework	Strengths	Weaknesses	Decision
React with Vite (SPA)	Simple mental model; fast development server; complete control of the bundle.	No server rendering, so first paint waits on JavaScript; routing, data loading and code splitting assembled by hand.	Rejected — first paint matters on the target devices.
Next.js (App Router)	Server and client components; route-level code splitting by default; image and font optimisation; mature ecosystem.	Framework conventions must be learned; some complexity in the server/client boundary.	Adopted.
Angular	Comprehensive and opinionated; strong typing throughout.	Larger baseline payload; heavier for a team already fluent in React.	Rejected — payload and team familiarity.
Nuxt (Vue)	Comparable capability to Next.js.	The team had no Vue experience; no offsetting advantage.	Rejected — no reason to pay the learning cost.

4.2.2 Packages and tools

Package	Purpose	Note
Tailwind CSS	Utility-first styling.	Logical properties throughout, which is what makes right-to-left mirroring automatic rather than a second stylesheet.
DaisyUI	Component layer over Tailwind.	Supplies the two named themes.
Framer Motion	Animation.	Loaded through LazyMotion with the DOM animation feature set only — see section 4.8.
Recharts	Dashboard and assessment charts.	Dynamically imported; excluded from first load.
Redux Toolkit	Client state and cached server data.	Used where state is shared across distant components.
Lucide React	Icon set.	Tree-shakeable; individual icons imported.
Monaco Editor	In-browser code editor.	Loaded only on the practice route.
React Toastify	Transient notifications.	
Socket.IO client	Realtime messaging and notifications.	Endpoint must be configured explicitly — see section 5.4.

4.2.3 Interface design principles

A) Bilingual layout

Arabic support was treated as a layout requirement rather than a translation exercise. Switching locale changes the document direction, and the layout follows because spacing uses logical properties — inline-start and inline-end — rather than fixed left and right. This means a single stylesheet serves both directions, and a new component is bidirectional by default rather than by remembering to make it so.

B) Theme

Two themes are provided through DaisyUI. The selection persists across sessions and applies at the document root, so no component carries theme conditionals.

C) Progressive web application

A manifest and service worker make the platform installable, with an offline application shell. This was chosen over native applications for the reason given in section 1.3.2.

4.2.4 User Interface Walkthrough

This section presents the delivered interface. Every screenshot was captured from the running application against a live database; none is a mockup. The learner shown was registered through the ordinary registration flow, and the roadmap was generated by the system at capture time.

4.2.4.1 Public pages

The entry point states the proposition and routes visitors to registration. The navigation bar carries the three controls that appear on every screen thereafter: the language toggle, the theme switch, and the installation prompt for the progressive web application.

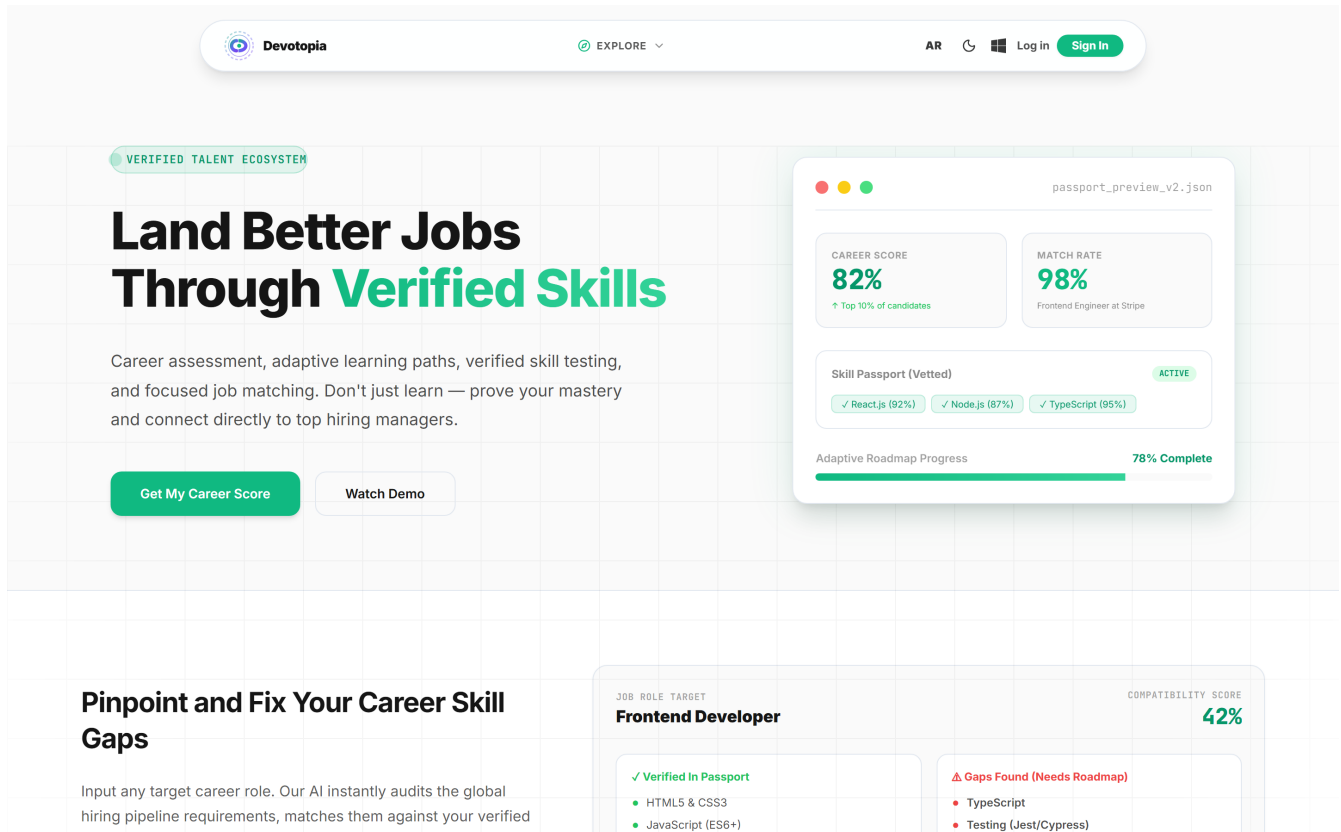
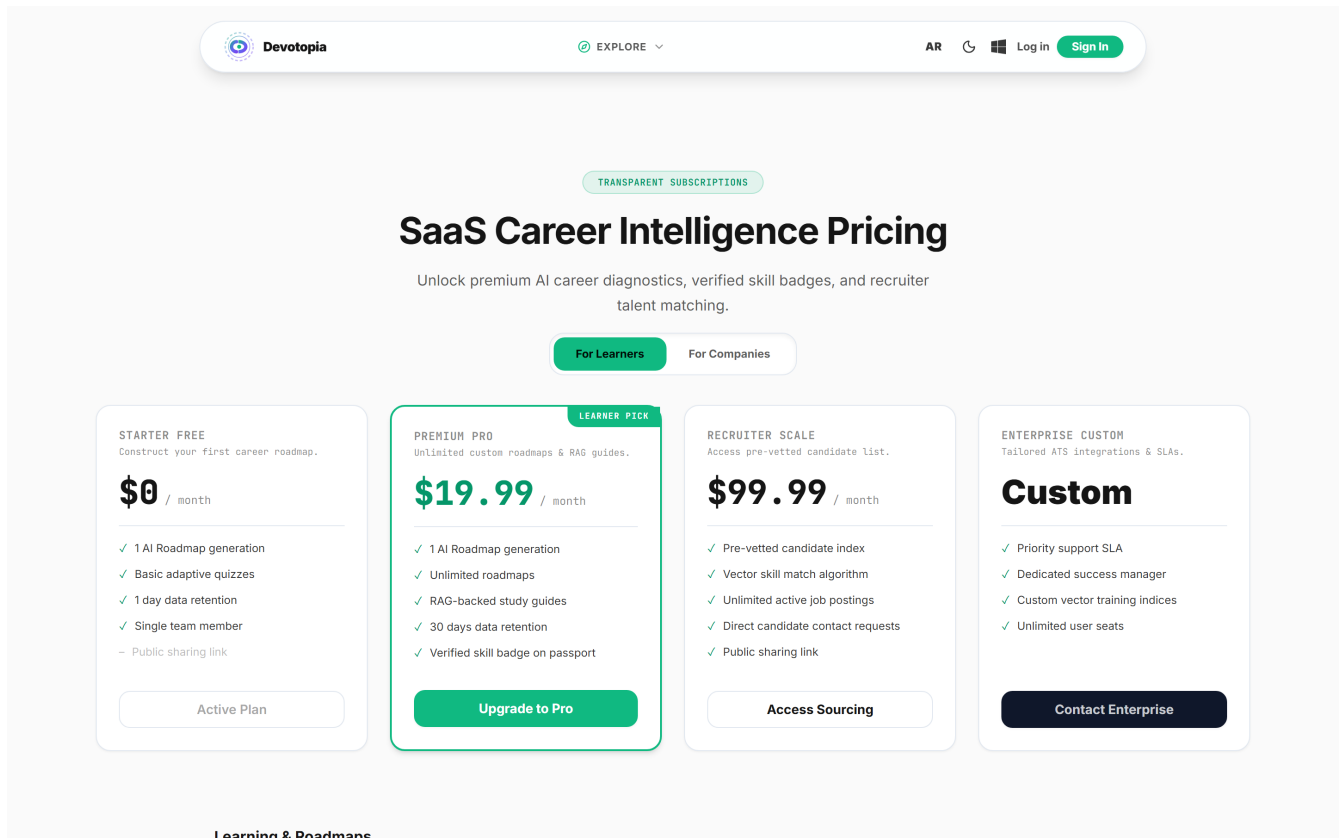


Figure 4.4 — Landing page. Language (AR), theme and install controls sit in the navigation bar.

The pricing page presents the three plans described in section 2.4. The mechanism behind it is more important than the presentation: the client sends only a plan identifier, and the amount is resolved server-side, so a tampered request cannot alter what is charged.



Devotopia

EXPLORE

AR

Log in

Sign in

TRANSPARENT SUBSCRIPTIONS

SaaS Career Intelligence Pricing

Unlock premium AI career diagnostics, verified skill badges, and recruiter talent matching.

For Learners For Companies

STARTER FREE
Construct your first career roadmap.

\$0 / month

- ✓ 1 AI Roadmap generation
- ✓ Basic adaptive quizzes
- ✓ 1 day data retention
- ✓ Single team member
- Public sharing link

Active Plan

PREMIUM PRO
Unlimited custom roadmaps & RAG guides.

\$19.99 / month

- ✓ 1 AI Roadmap generation
- ✓ Unlimited roadmaps
- ✓ RAG-backed study guides
- ✓ 30 days data retention
- ✓ Verified skill badge on passport

Upgrade to Pro

RECRUITER SCALE
Access pre-vetted candidate list.

\$99.99 / month

- ✓ Pre-vetted candidate index
- ✓ Vector skill match algorithm
- ✓ Unlimited active job postings
- ✓ Direct candidate contact requests
- ✓ Public sharing link

Access Sourcing

ENTERPRISE CUSTOM
Tailored ATS integrations & SLAs.

Custom

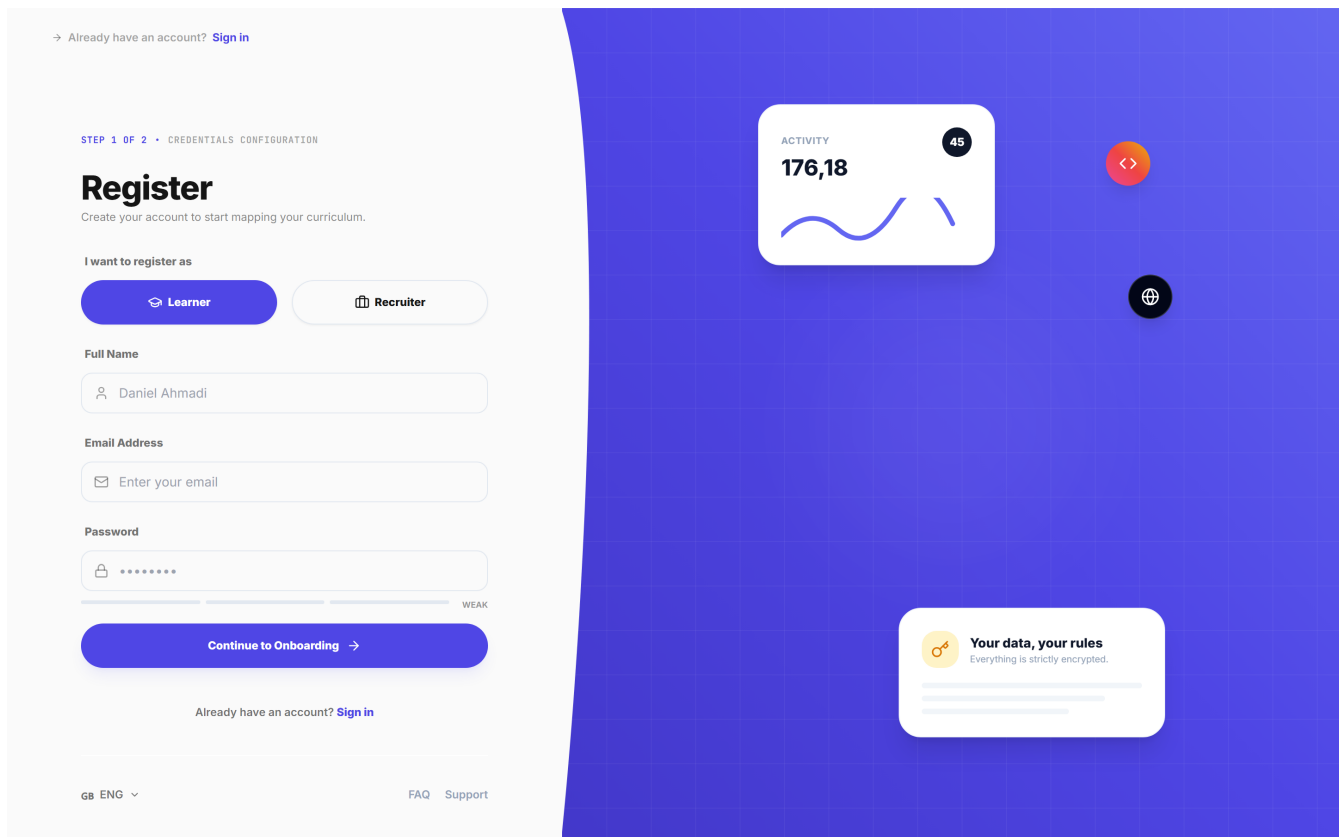
- ✓ Priority support SLA
- ✓ Dedicated success manager
- ✓ Custom vector training indices
- ✓ Unlimited user seats

Contact Enterprise

Learning & Roadmaps

Figure 4.5 — Pricing. Amounts are resolved server-side from the plan identifier.

Registration is a two-step form. The first step collects credentials and validates them client-side; the second collects the profile information that becomes input to curriculum generation. Both steps report failure identically, which is a deliberate simplification but one with a diagnostic cost noted in section 5.4.



→ Already have an account? [Sign in](#)

STEP 1 OF 2 • CREDENTIALS CONFIGURATION

Register

Create your account to start mapping your curriculum.

I want to register as

☒ Learner ☐ Recruiter

Full Name

Email Address

Password

WEAK

[Continue to Onboarding →](#)

Already have an account? [Sign in](#)

GB ENG ▾

[FAQ](#) [Support](#)

ACTIVITY

176,18

45

<>

🌐

 **Your data, your rules**
Everything is strictly encrypted.

Figure 4.6 — Registration, step one. The role selector determines which second step is presented.

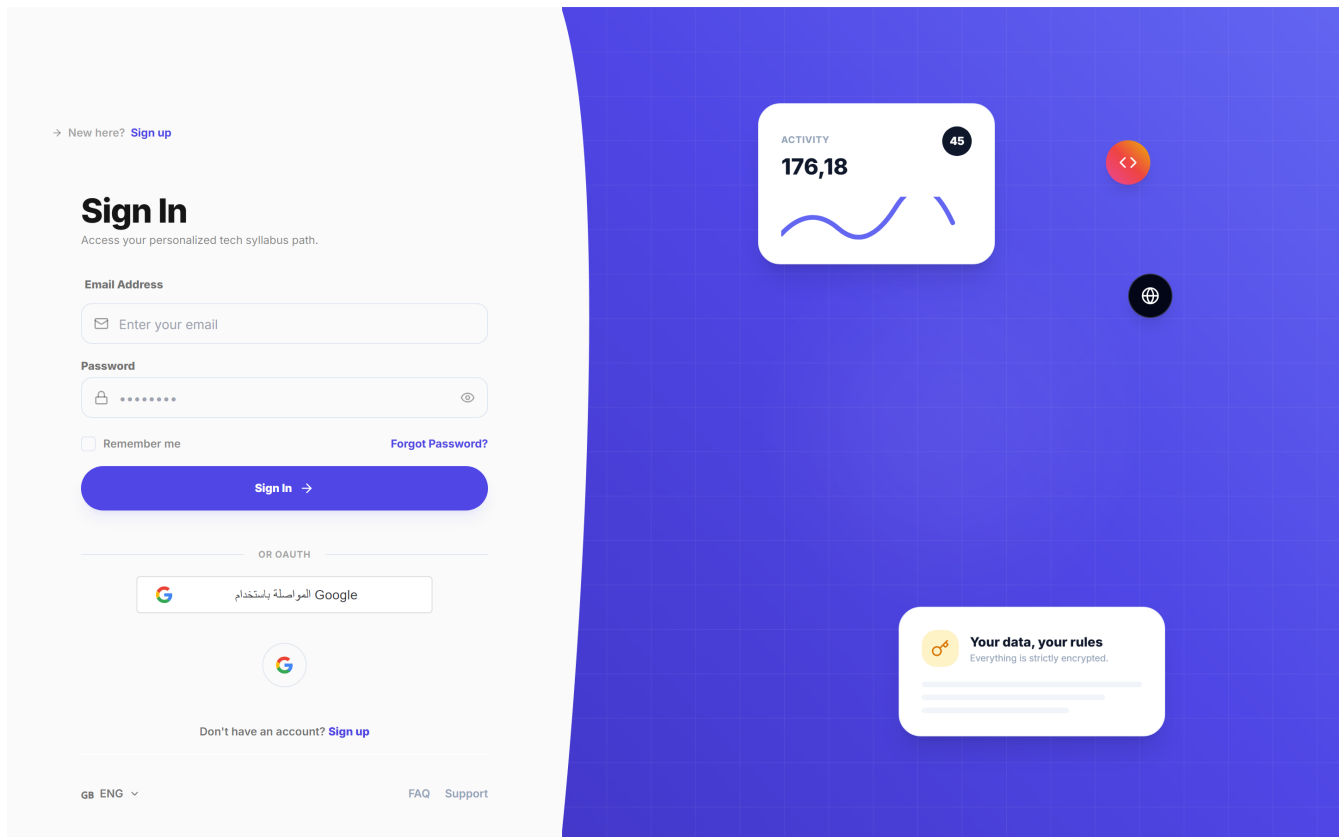
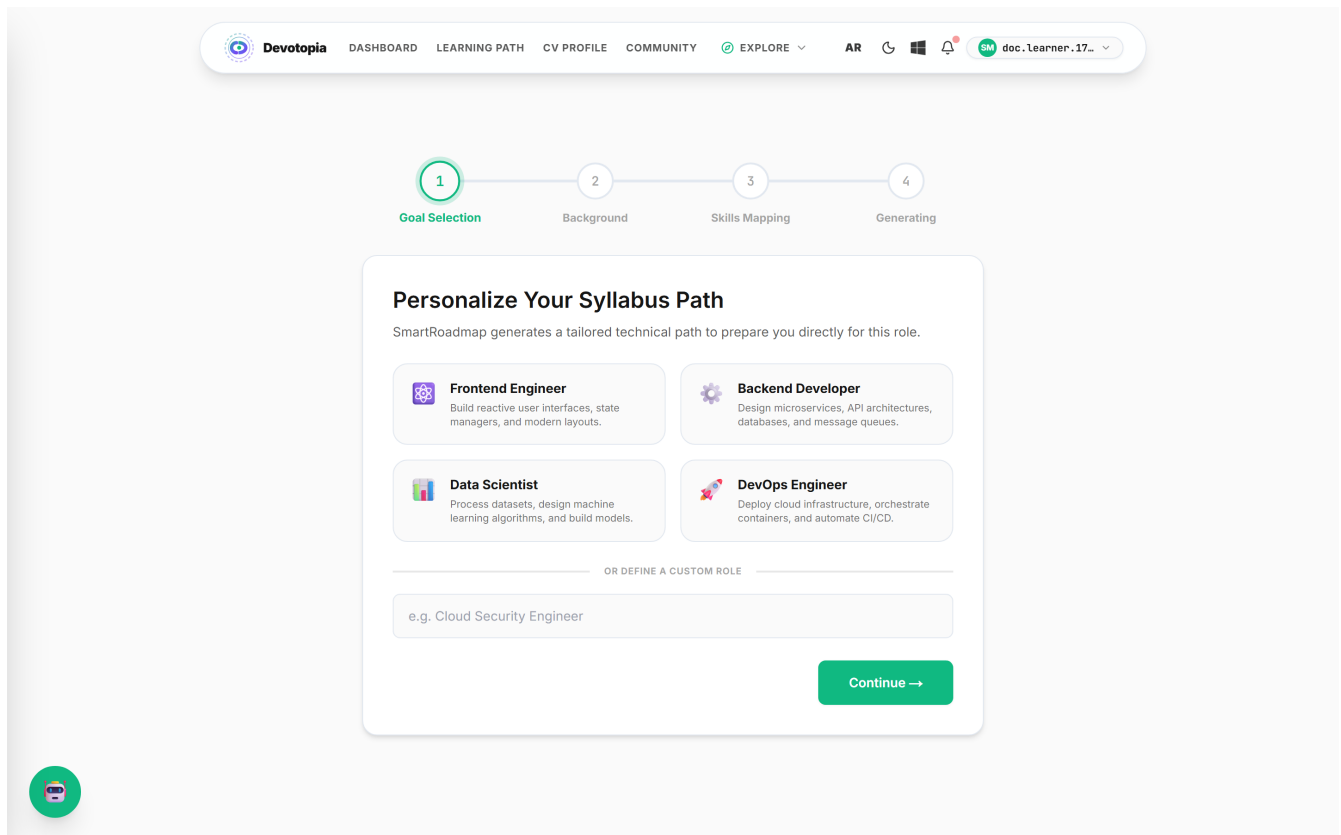


Figure 4.7 — Sign in. The same screen handles Google identity sign-in.

4.2.4.2 The learner interface

Onboarding collects the four inputs the generator requires — target role, education, years of experience and current skills. Nothing further is requested, because nothing further changes the output.



The screenshot shows the Devotopia onboarding interface. At the top is a navigation bar with the Devotopia logo and links for DASHBOARD, LEARNING PATH, CV PROFILE, COMMUNITY, EXPLORE, and user settings. Below the navigation bar is a progress indicator with four steps: 1. Goal Selection (active), 2. Background, 3. Skills Mapping, and 4. Generating. The main content area is titled 'Personalize Your Syllabus Path' and includes a sub-header: 'SmartRoadmap generates a tailored technical path to prepare you directly for this role.' There are four role selection cards: Frontend Engineer (Build reactive user interfaces, state managers, and modern layouts.), Backend Developer (Design microservices, API architectures, databases, and message queues.), Data Scientist (Process datasets, design machine learning algorithms, and build models.), and DevOps Engineer (Deploy cloud infrastructure, orchestrate containers, and automate CI/CD.). Below these cards is a section 'OR DEFINE A CUSTOM ROLE' with a text input field containing 'e.g. Cloud Security Engineer' and a 'Continue →' button.

Figure 4.8 — Onboarding. These four answers form the generation prompt.

After generation the learner arrives at the dashboard. Roadmap progress, the learning streak, assessment history and study activity are all live queries against recorded data; the streak and achievement values are written by the same code path that completes an assessment, so what is displayed here is the direct product of the flow described in section 4.5.

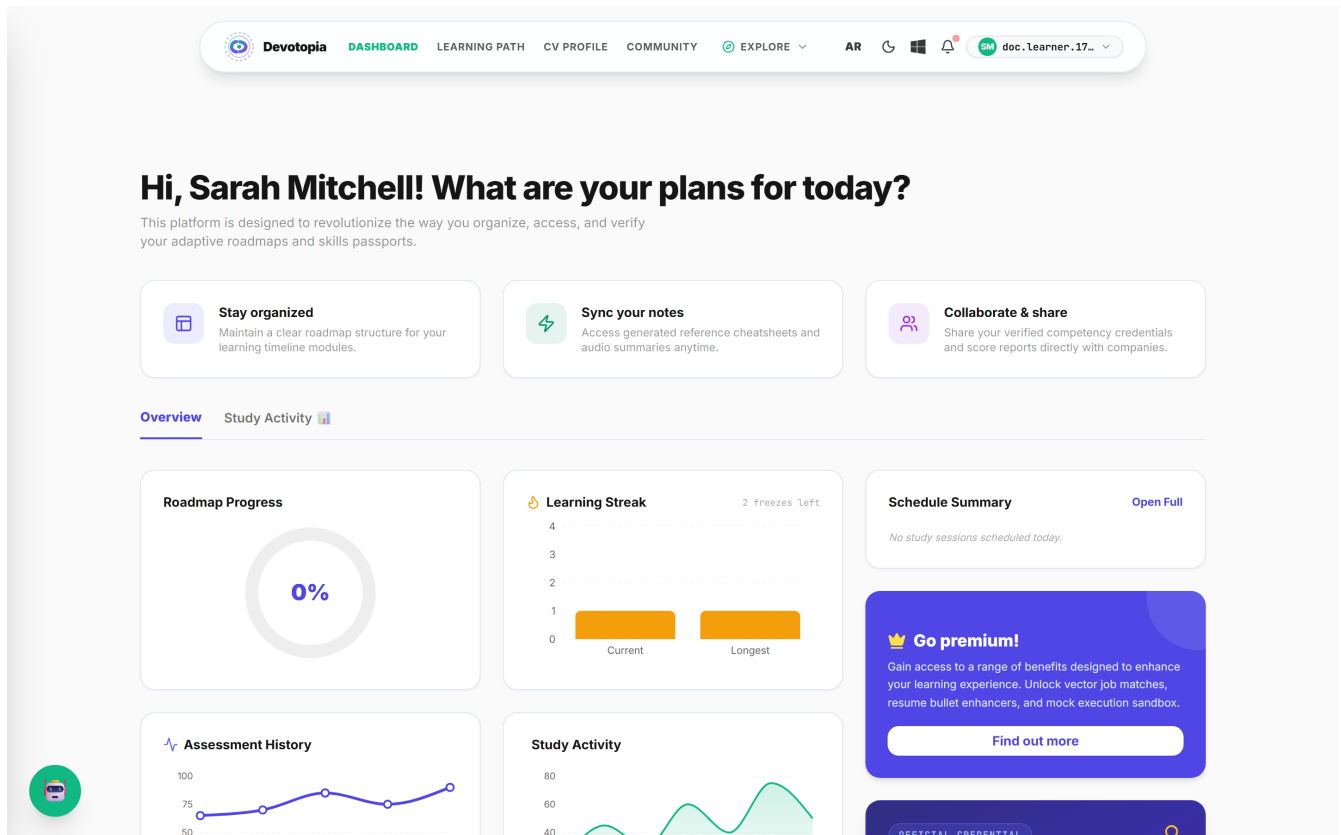


Figure 4.9 — Learner dashboard. Charts are rendered from recorded assessment and activity data.

The complete dashboard, showing the credential card and recommended next actions below the fold:

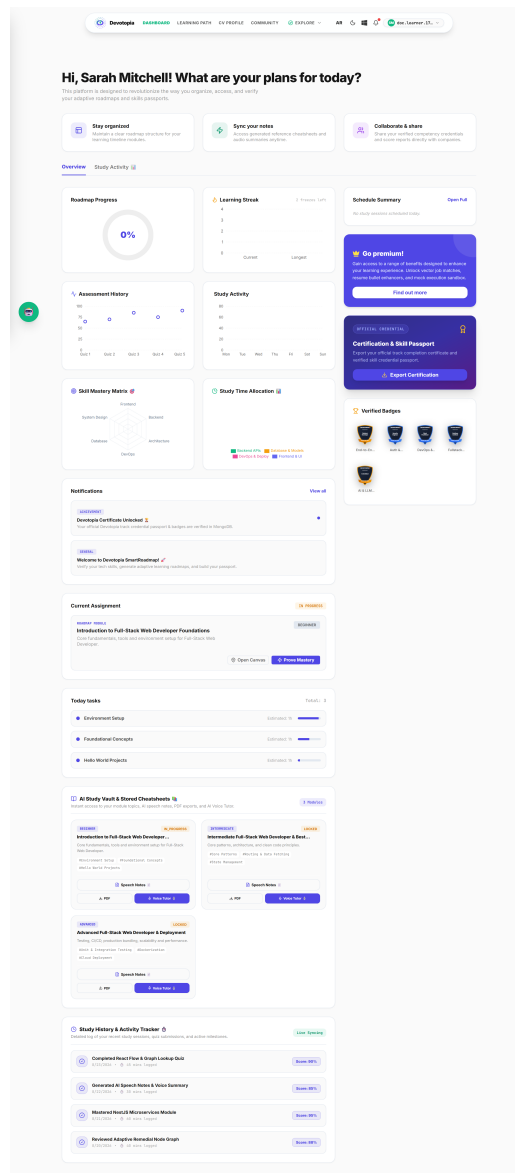


Figure 4.10 — Full dashboard view.

The roadmap renders the generated curriculum as a tree. Each node carries a difficulty tier, an hour estimate and a state — locked, in progress, completed or failed. The layout coordinates are produced by the generator itself, so the shape of the tree is part of the generated data rather than a heuristic applied in the browser.

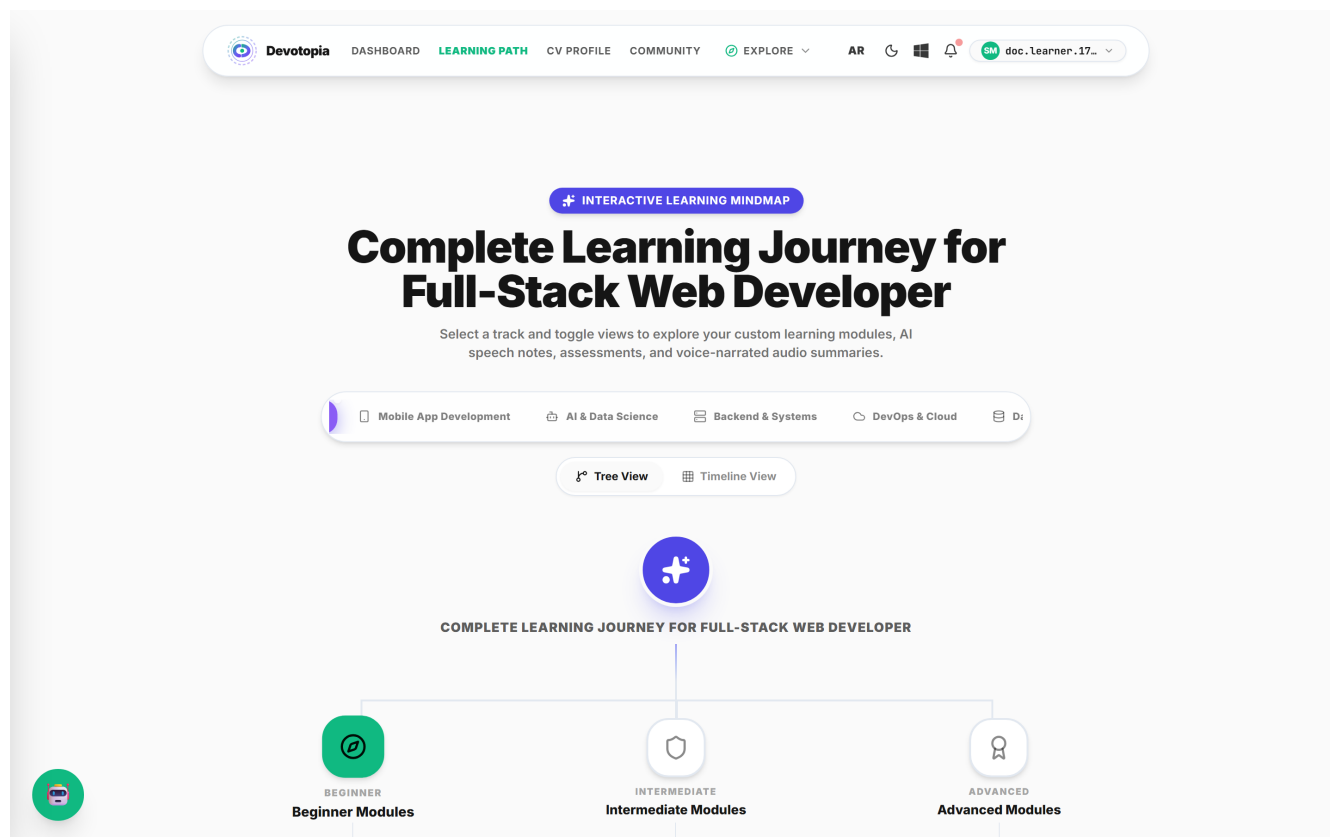


Figure 4.11 — Learning path. Branches by difficulty tier, with per-node state.

4.2.4.3 The assessment interface

This is the screen where the mechanism of section 4.5 becomes visible to the learner. Three elements are worth noting: the question counter, the difficulty badge showing the level at which this question was posed, and the countdown. The difficulty badge is not decoration — it is the value that will weight this answer in the final score.

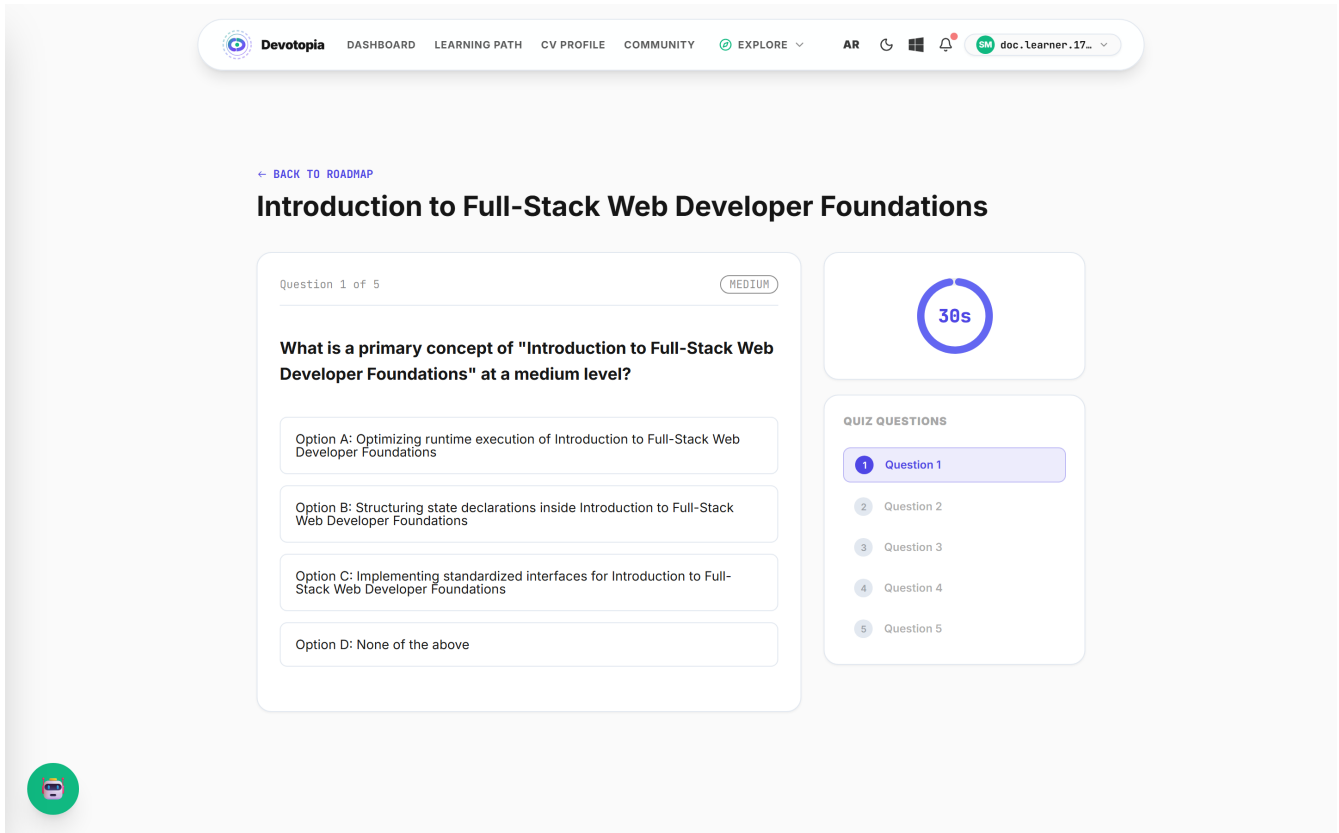
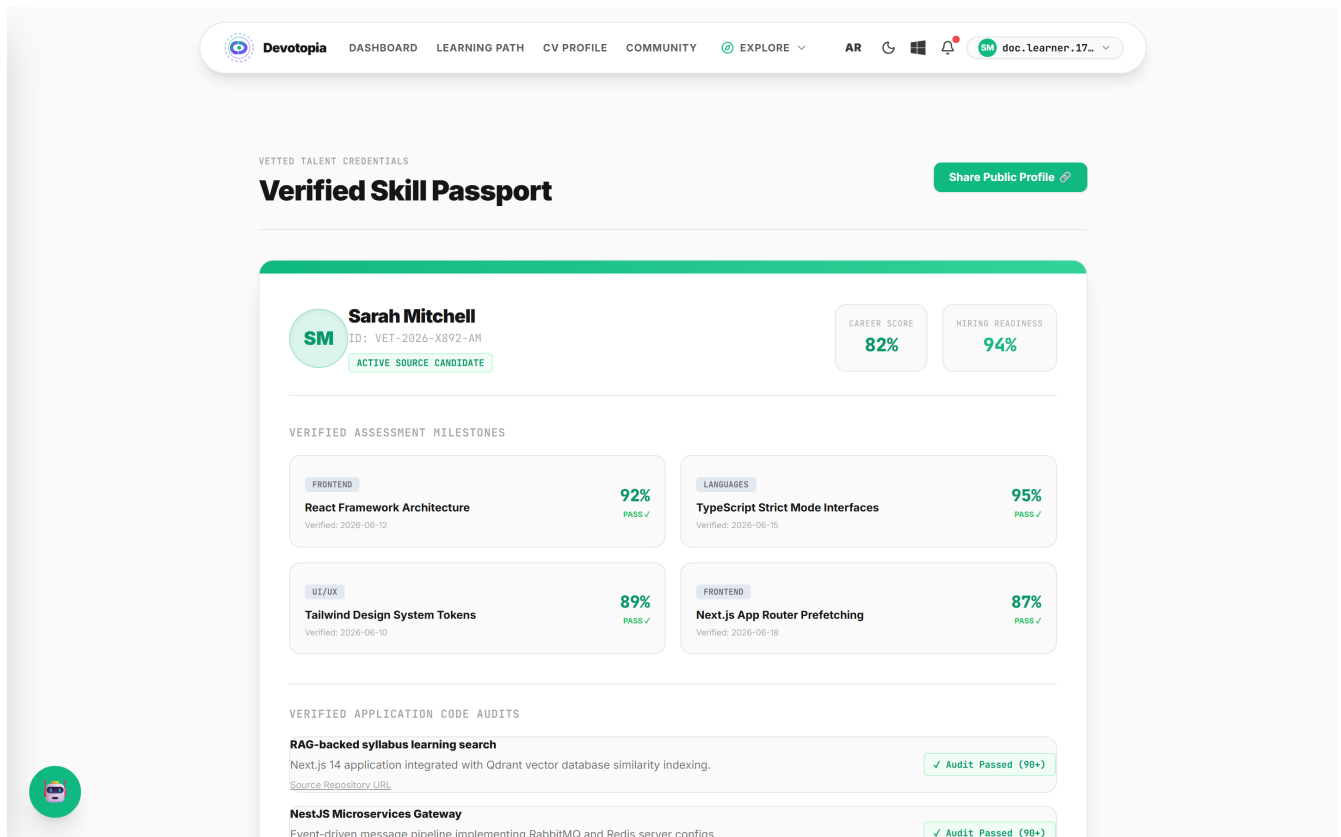


Figure 4.12 — Assessment. The MEDIUM badge is the weight this answer will carry; the ring is the 30-second limit.

The countdown ring is a plain SVG element rather than a charting component. It was originally rendered with the charting library, which pulled approximately 200 kB into the route to draw two arcs; the replacement is described in section 4.8.2.

4.2.4.4 Verified profile and hiring

Passed assessments accumulate into the skill passport — the artefact a recruiter reads. Each verified milestone carries the score achieved and the date of verification, and the record is shareable as a public profile.



Devotopia DASHBOARD LEARNING PATH CV PROFILE COMMUNITY EXPLORE AR doc..learner.17..

VETTED TALENT CREDENTIALS

Verified Skill Passport

Share Public Profile

SM Sarah Mitchell
ID: VET-2026-X892-AM
ACTIVE SOURCE CANDIDATE

CAREER SCORE **82%** HIRING READINESS **94%**

VERIFIED ASSESSMENT MILESTONES

Category	Assessment	Score	Status	Verified
FRONTEND	React Framework Architecture	92%	PASS ✓	2026-06-12
LANGUAGES	TypeScript Strict Mode Interfaces	95%	PASS ✓	2026-06-15
UI/UX	Tailwind Design System Tokens	89%	PASS ✓	2026-06-10
FRONTEND	Next.js App Router Prefetching	87%	PASS ✓	2026-06-18

VERIFIED APPLICATION CODE AUDITS

Project	Description	Status
RAG-backed syllabus learning search	Next.js 14 application integrated with Qdrant vector database similarity indexing. Source Repository URL	✓ Audit Passed (90+)
NestJS Microservices Gateway	Event-driven message pipeline implementing RabbitMQ and Redis server configs.	✓ Audit Passed (90+)

Figure 4.13 — Verified skill passport, with career score and hiring readiness.

The CV builder parses an uploaded résumé into structured sections rather than storing free text, which is what allows individual experience statements to be enhanced independently while the original is preserved.

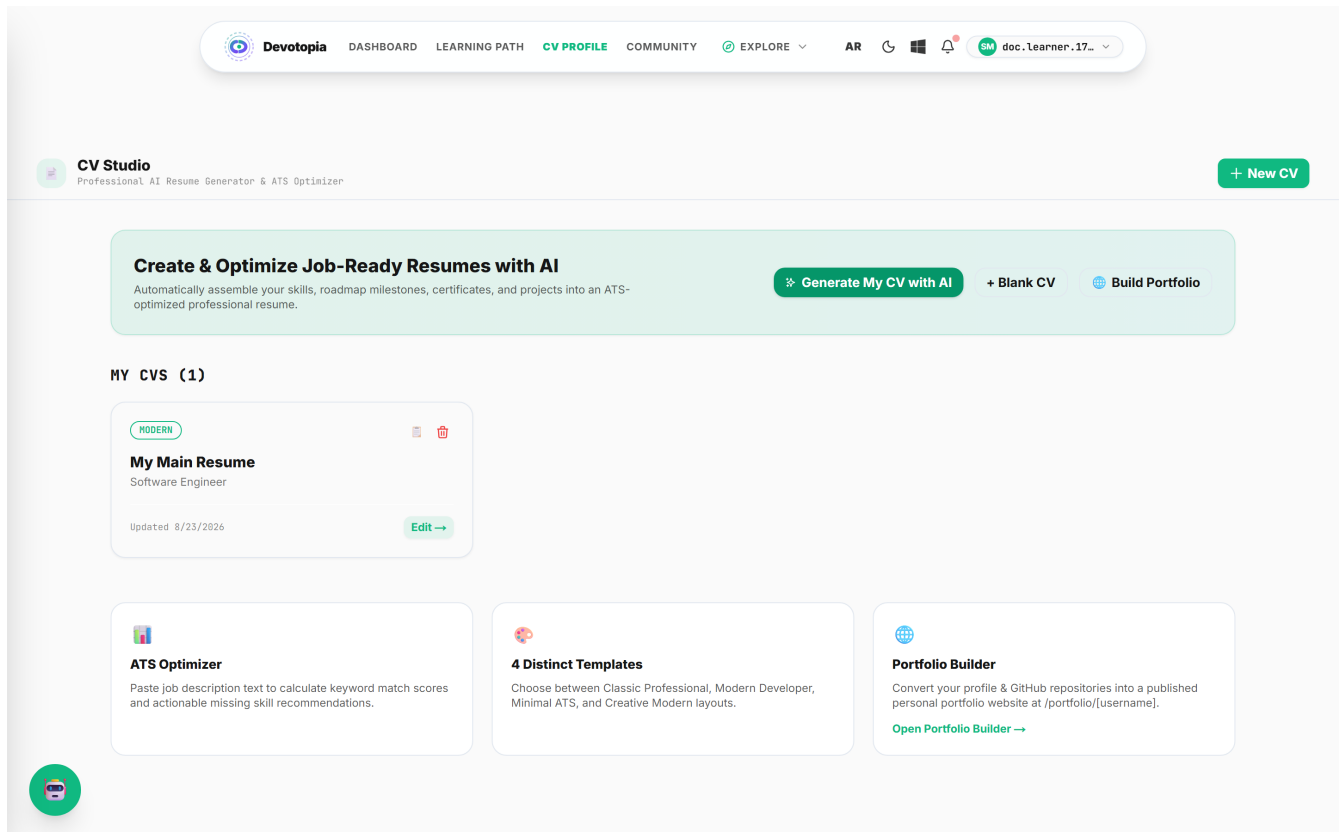
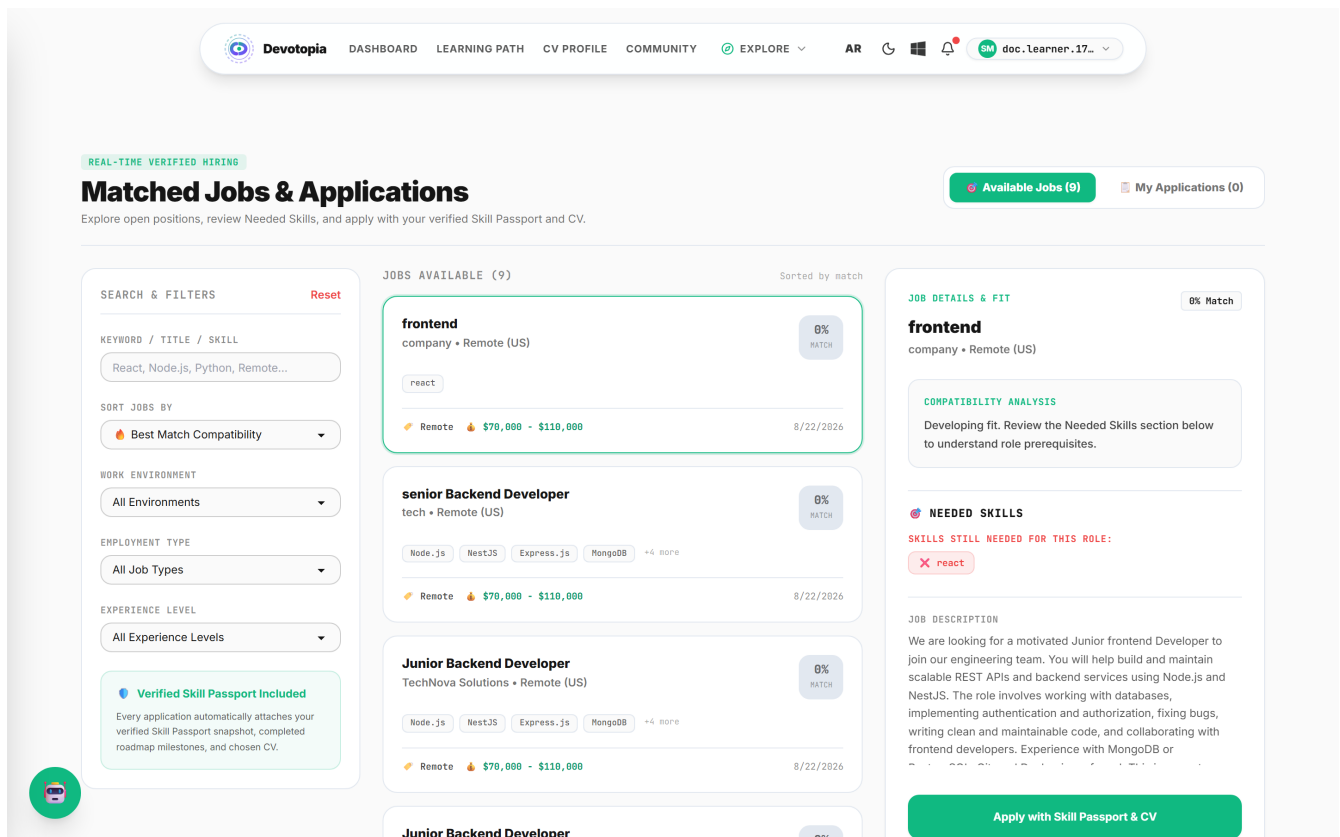


Figure 4.14 — CV editor. Structured sections rather than a single text field.

Job matching compares verified skills against a posting's requirements and presents the difference. Where a requirement is missing, it can be injected back into the roadmap as new modules — the action that closes the loop between hiring and learning.



The screenshot displays the Devotopia user interface for job matching. The top navigation bar includes links to Dashboard, Learning Path, CV Profile, Community, and Explore, along with a user profile dropdown for 'doc.Learner.17...'. The main section is titled 'Matched Jobs & Applications' with a subtitle 'Explore open positions, review Needed Skills, and apply with your verified Skill Passport and CV.' It shows 'Available Jobs (9)' and 'My Applications (0)'. On the left, a 'SEARCH & FILTERS' sidebar allows filtering by keyword (React, Node.js, Python, Remote...), sort order (Best Match Compatibility), work environment (All Environments), employment type (All Job Types), and experience level (All Experience Levels). A 'Verified Skill Passport Included' badge is also present. The main area lists three jobs: 'frontend', 'senior Backend Developer', and 'Junior Backend Developer', all with a 0% match. The 'frontend' job details are expanded on the right, showing a 'COMPATIBILITY ANALYSIS' section with the text 'Developing fit. Review the Needed Skills section below to understand role prerequisites.' and a 'NEEDED SKILLS' section listing 'react' as a skill still needed for this role. A green button at the bottom right of the job details says 'Apply with Skill Passport & CV'.

Figure 4.15 — Job matching with gap analysis.

4.2.4.5 Supporting surfaces

Beyond the core loop, the platform provides the surfaces a learner uses day to day. Each is backed by a full feature module on the server rather than being a static page.

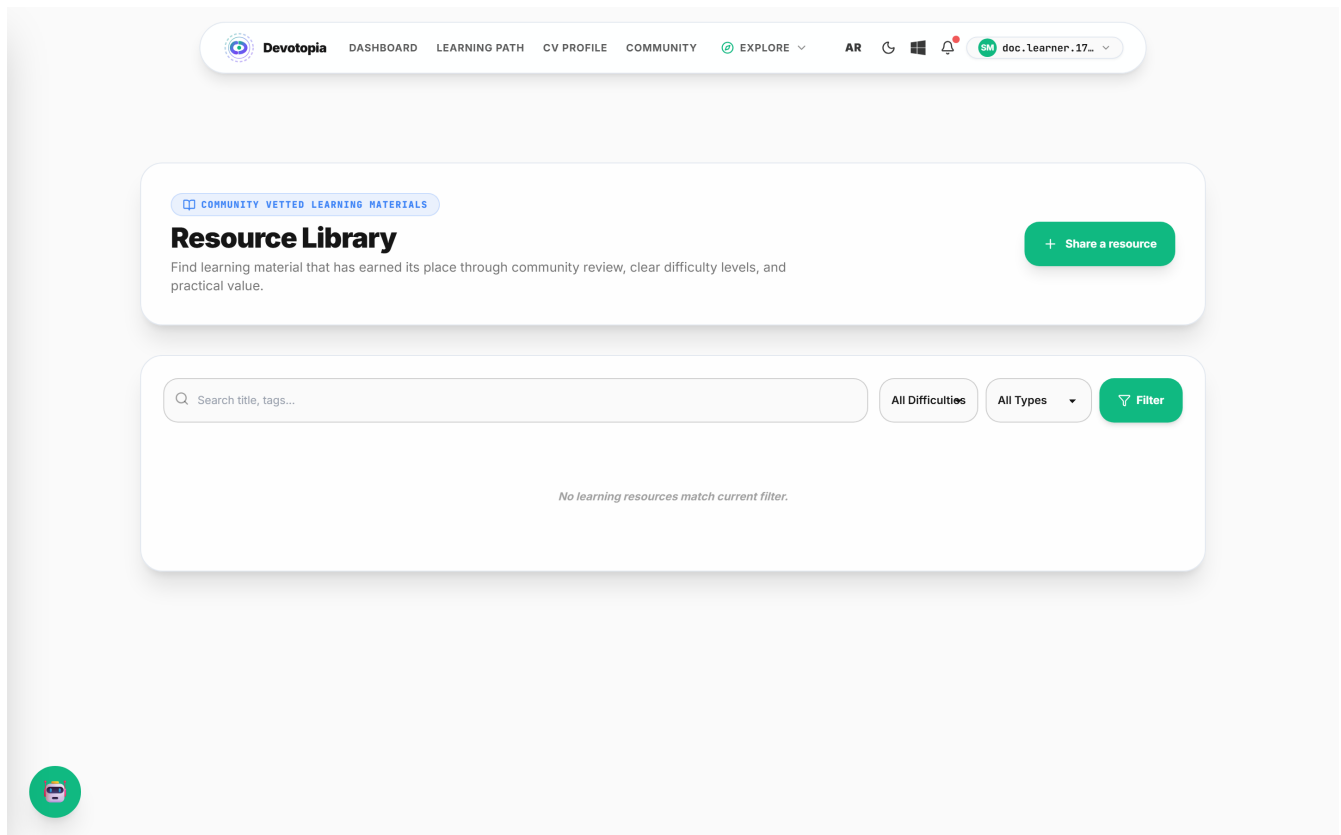


Figure 4.16 — Learning resources, retrieved by semantic search over embedded content.

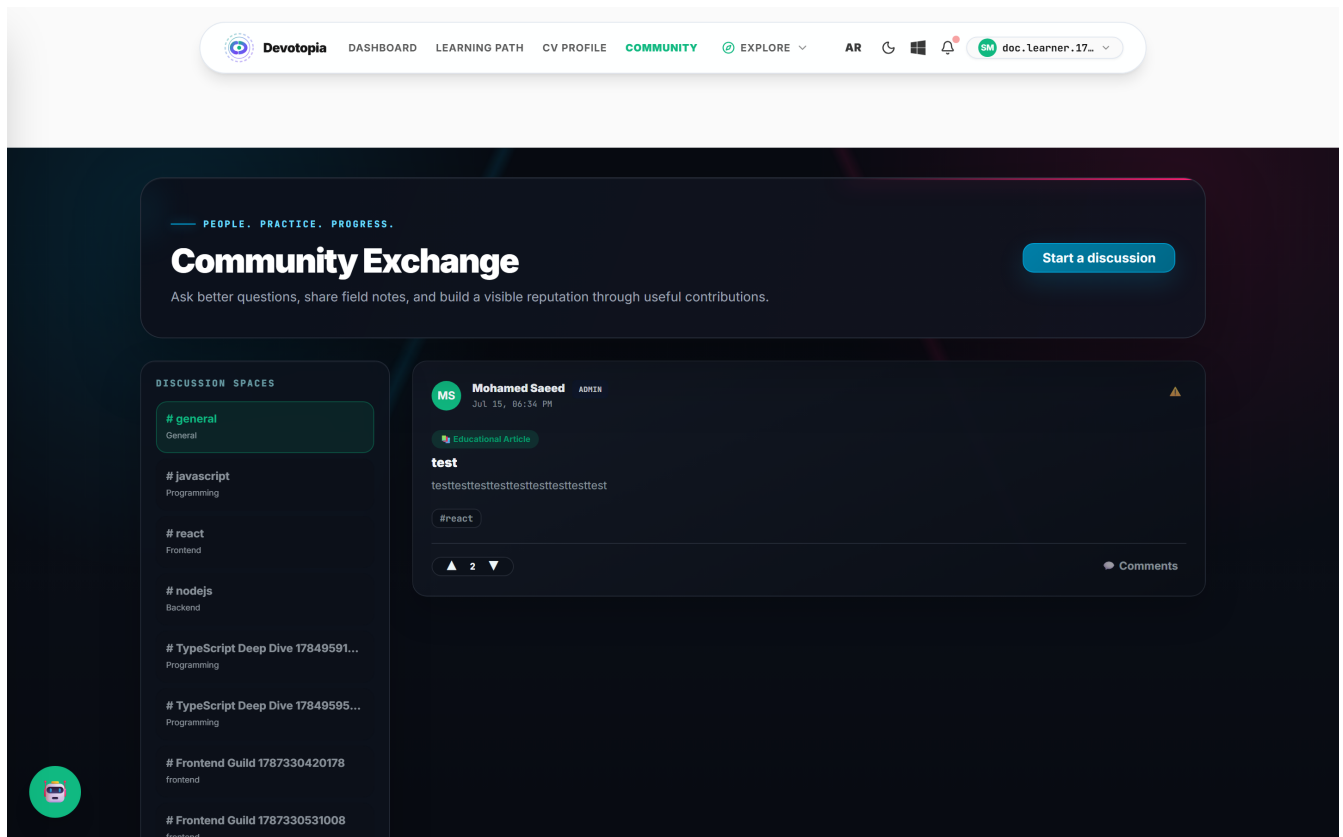


Figure 4.17 — Community discussion spaces.


Devotopia
DASHBOARD
LEARNING PATH
CV PROFILE
COMMUNITY
EXPLORE
AR
doc. learner.17...

DIRECT CAREER & TECHNICAL MENTORING

Mentor Network

Work directly with experienced practitioners, unblock difficult decisions, and keep your learning plan moving.

[Join the network](#)

CURATED
Recommended for your current path
MATCHED BY SKILLS, AVAILABILITY, AND LEARNER GOALS


Mohamed Saeed
SOFTWARE • 3 YEARS EXP
Expertise in react matches your skill development paths.
[Book session](#)

40% Match


CM mentor
SOFTWARE ENGINEERING • 10 YEARS EXP
Expertise in React, NestJS, MongoDB matches your skill development paths.
[Book session](#)

40% Match

YOUR BOOKED MENTORSHIP SESSIONS

No scheduled sessions found.



[Search](#)

Figure 4.18 — Mentor directory with ratings and session booking.

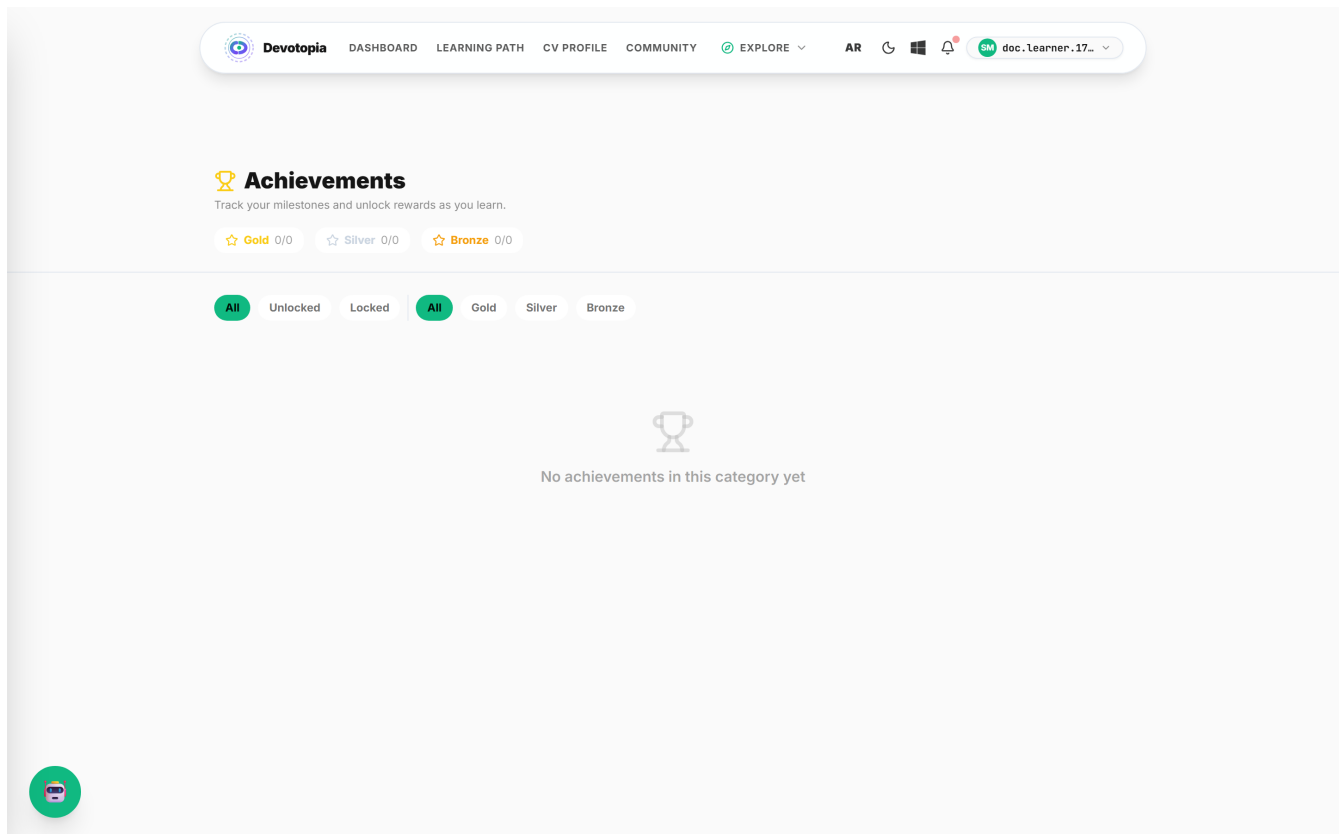


Figure 4.19 — Achievements. Twenty-six definitions are seeded at startup and awarded by event listeners.

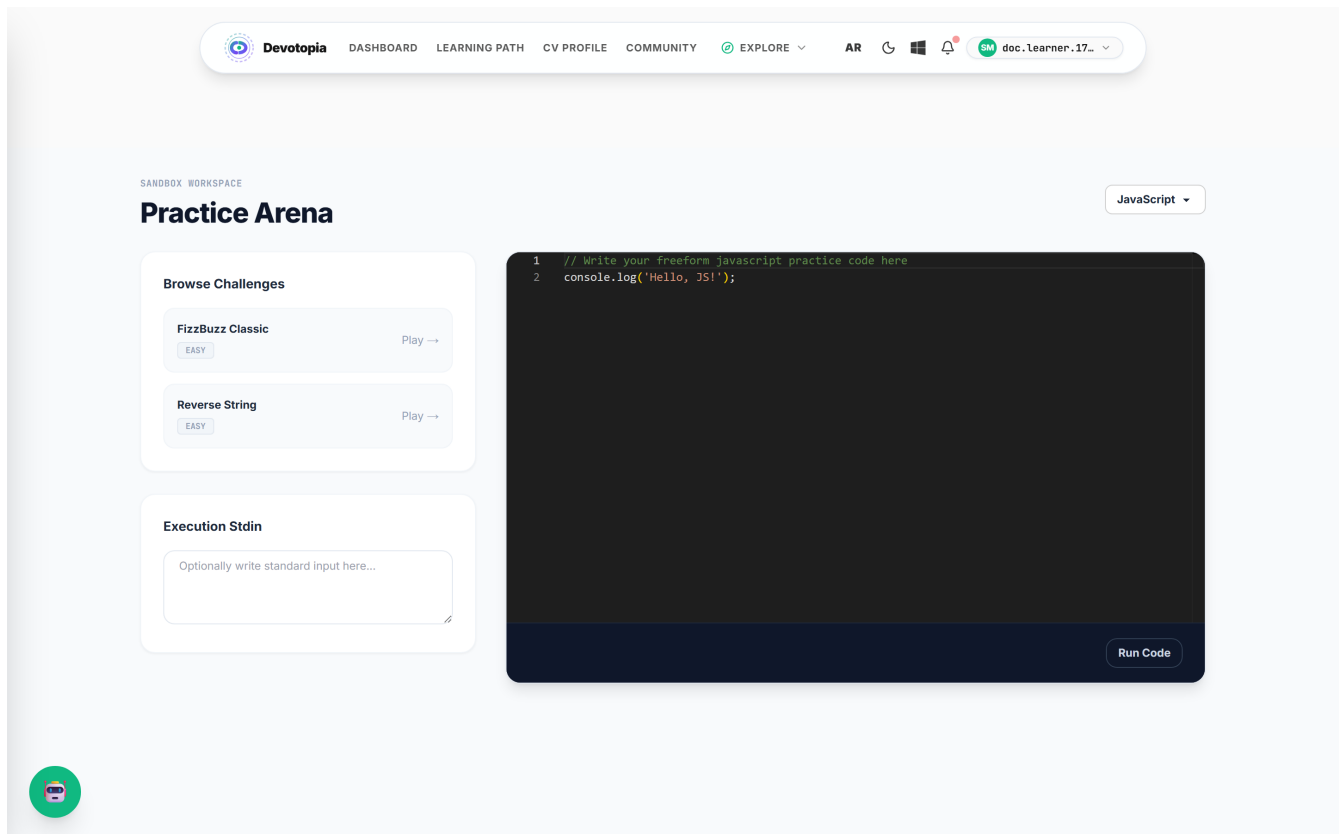


Figure 4.20 — Coding practice, executed in an isolated sandbox.


Devotopia

[DASHBOARD](#)
[LEARNING PATH](#)
[CV PROFILE](#)
[COMMUNITY](#)
[EXPLORE](#)

AR

SM doc. learner.17...



Sarah Mitchell
SYLLABUS LEARNER

[Message](#)
[Follow](#)

July 2026

S	M	T	W	T	F	S
28	29	30	1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25

REVIEWS



Monica Richard ★ 4.5

Biology curriculum path generated with excellent RAG citation matching.



Sarah Blue ★ 4.6

Mastered JavaScript in record time using custom sandbox execution verification.

SUBJECTS

Math

Music

Biology

Foundations

Study Slots

Choose hours available for syllabus auto-scheduling.

☐ Mon
 ☐ Tue
 ☐ Wed
 ☐ Thu

Current week schedule

Click Auto-Schedule to align active roadmap modules to weekly slots.

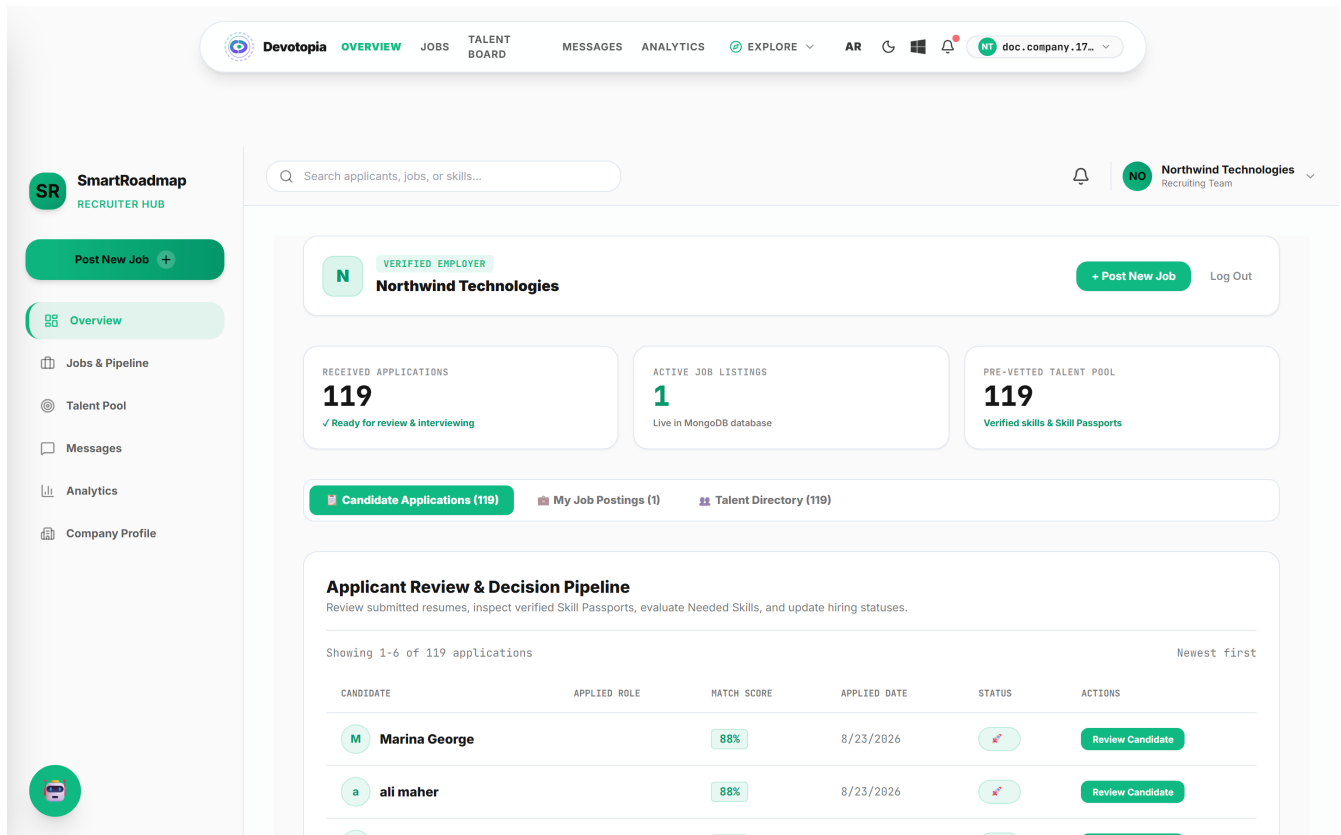
[Add Event +](#)
[Auto-Schedule Study Plan](#)

Days	13:00	14:00	15:00	16:00	17:00	18:00	19:00	20:00
Mon								
Tue								
Wed								
Thu								
Fri								

Figure 4.21 — Study calendar and scheduled sessions.

4.2.4.6 The employer interface

A company account sees a different application from the same deployment. The separation is enforced on the server, not in the browser: navigating to a learner route with a company session returns a refusal rather than a rendered page, which was verified directly during testing.



The screenshot shows the 'SmartRoadmap RECRUITER HUB' interface for a company named 'Northwind Technologies'. The top navigation bar includes links for OVERVIEW, JOBS, TALENT BOARD, MESSAGES, ANALYTICS, EXPLORE, and AR. A search bar is located below the navigation. The left sidebar contains a menu with 'Overview' (selected), 'Jobs & Pipeline', 'Talent Pool', 'Messages', 'Analytics', and 'Company Profile'. The main content area displays a 'VERIFIED EMPLOYER' badge for Northwind Technologies, along with buttons for '+ Post New Job' and 'Log Out'. Three key metrics are shown: 'RECEIVED APPLICATIONS' (119, Ready for review & interviewing), 'ACTIVE JOB LISTINGS' (1, Live in MongoDB database), and 'PRE-VETTED TALENT POOL' (119, Verified skills & Skill Passports). Below these, a summary bar shows 'Candidate Applications (119)', 'My Job Postings (1)', and 'Talent Directory (119)'. The 'Applicant Review & Decision Pipeline' section provides instructions on reviewing resumes and evaluating skills. It shows a table of the first 6 of 119 applications, sorted by 'Newest first'.

CANDIDATE	APPLIED ROLE	MATCH SCORE	APPLIED DATE	STATUS	ACTIONS
M Marina George		88%	8/23/2026		<button>Review Candidate</button>
a ali maher		88%	8/23/2026		<button>Review Candidate</button>

Figure 4.22 — Company dashboard.


Devotopia

[OVERVIEW](#)
[JOBS](#)
[TALENT BOARD](#)
[MESSAGES](#)
[ANALYTICS](#)
[EXPLORE](#)
[AR](#)





 doc.company.17...

REAL-TIME VERIFIED HIRING

Matched Jobs & Applications

Explore open positions, review Needed Skills, and apply with your verified Skill Passport and CV.

Available Jobs (9)

My Applications (0)

SEARCH & FILTERS

Reset

KEYWORD / TITLE / SKILL

React, Node.js, Python, Remote...

SORT JOBS BY

Best Match Compatibility

WORK ENVIRONMENT

All Environments

EMPLOYMENT TYPE

All Job Types

EXPERIENCE LEVEL

All Experience Levels

Verified Skill Passport Included

Every application automatically attaches your verified Skill Passport snapshot, completed roadmap milestones, and chosen CV.

JOBS AVAILABLE (9)

Sorted by match

frontend

company • Remote (US)

0% MATCH

React

Remote 🔥 \$70,000 - \$110,000

8/22/2026

senior Backend Developer

tech • Remote (US)

0% MATCH

Node.js NestJS Express.js MongoDB +4 more

Remote 🔥 \$70,000 - \$110,000

8/22/2026

Junior Backend Developer

TechNova Solutions • Remote (US)

0% MATCH

Node.js NestJS Express.js MongoDB +4 more

Remote 🔥 \$70,000 - \$110,000

8/22/2026

Junior Backend Developer

0% MATCH

JOB DETAILS & FIT

0% Match

frontend

company • Remote (US)

COMPATIBILITY ANALYSIS

Developing fit. Review the Needed Skills section below to understand role prerequisites.

NEEDED SKILLS

SKILLS STILL NEEDED FOR THIS ROLE:


 react

JOB DESCRIPTION

We are looking for a motivated Junior frontend Developer to join our engineering team. You will help build and maintain scalable REST APIs and backend services using Node.js and NestJS. The role involves working with databases, implementing authentication and authorization, fixing bugs, writing clean and maintainable code, and collaborating with frontend developers. Experience with MongoDB or

Apply with Skill Passport & CV

Figure 4.23 — Candidate pipeline, ranked by verified assessment performance.

Devotopia — Project Documentation

46

4.2.4.7 Theme, language and viewport

The same dashboard is shown below in dark mode and in Arabic. Switching locale mirrors the entire layout, including the roadmap tree, because spacing uses logical properties throughout rather than a second right-to-left stylesheet.

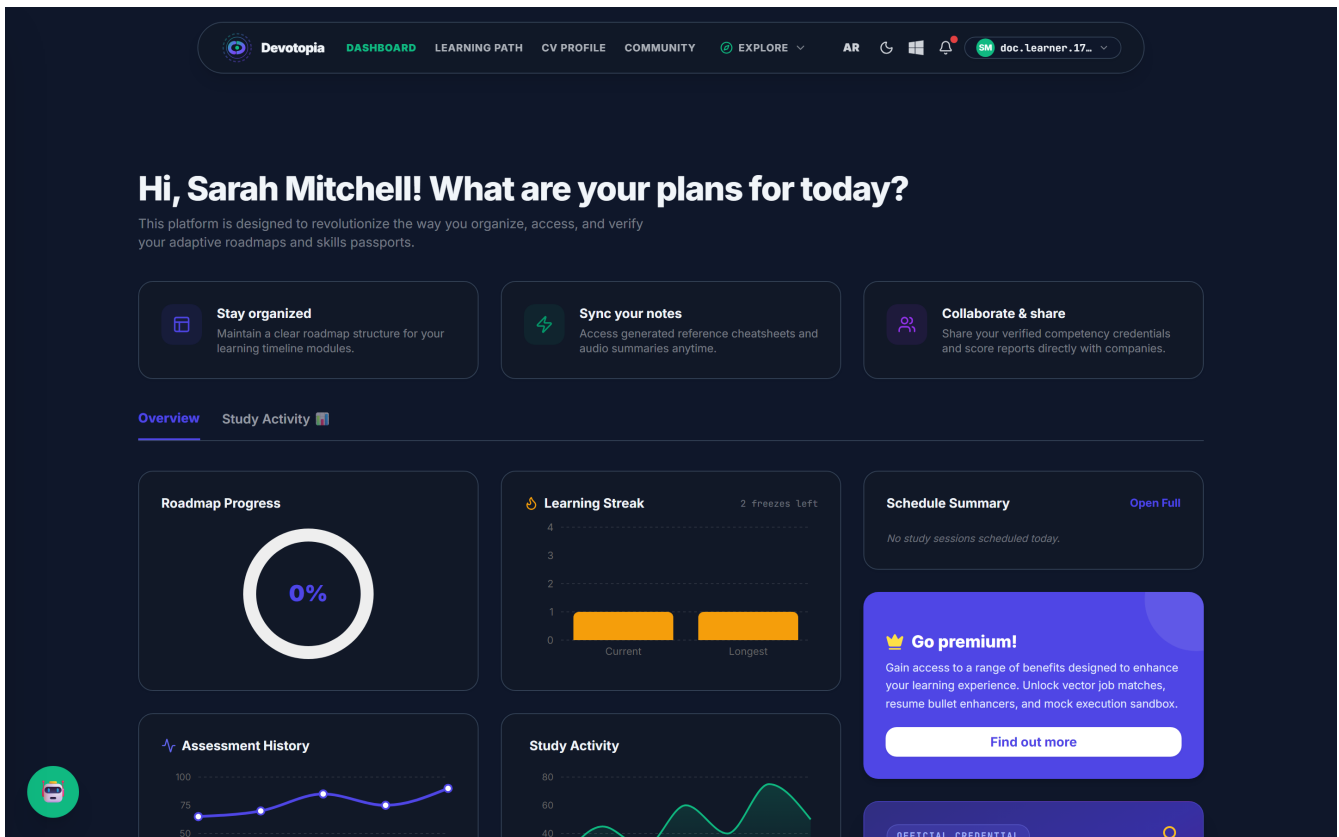


Figure 4.24 — Dashboard in dark theme.

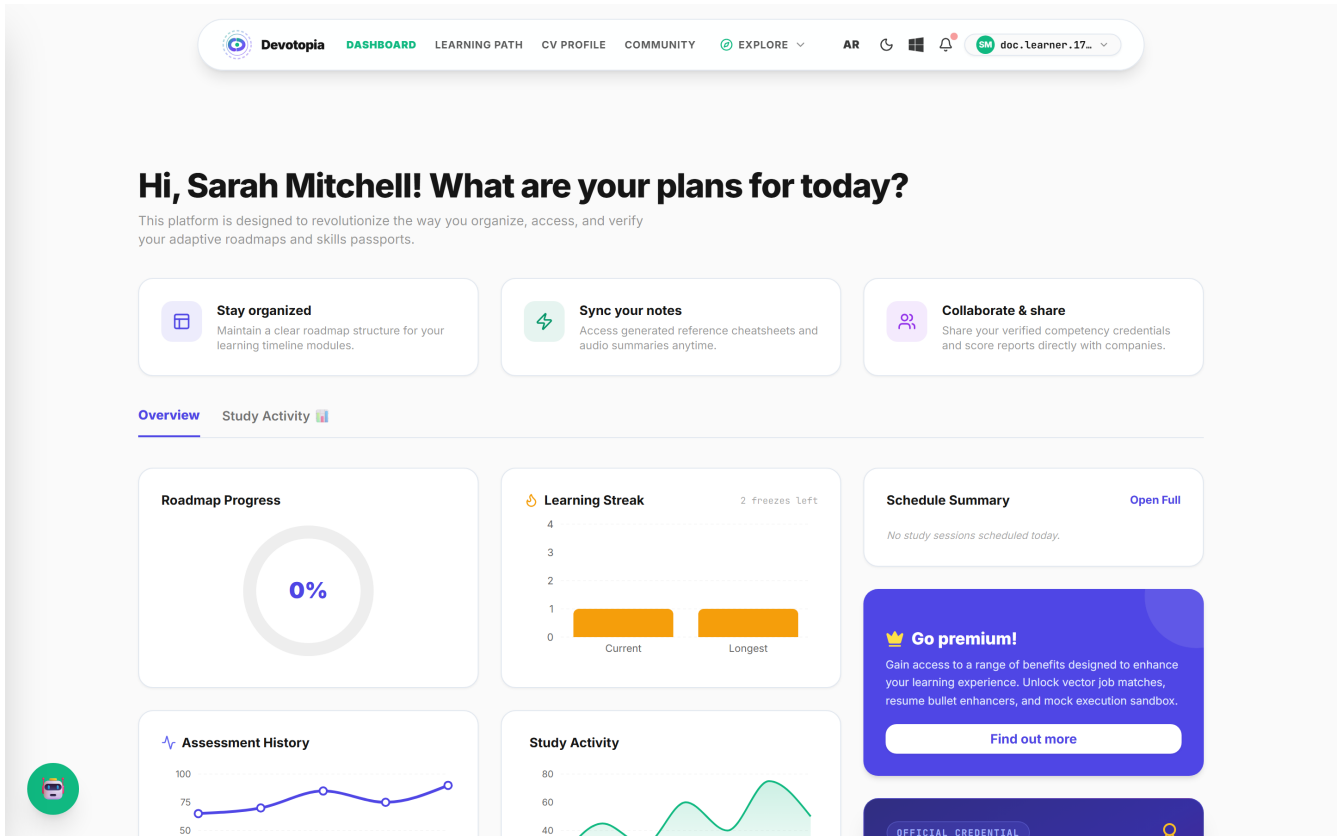


Figure 4.25 — The same dashboard in Arabic, right-to-left.

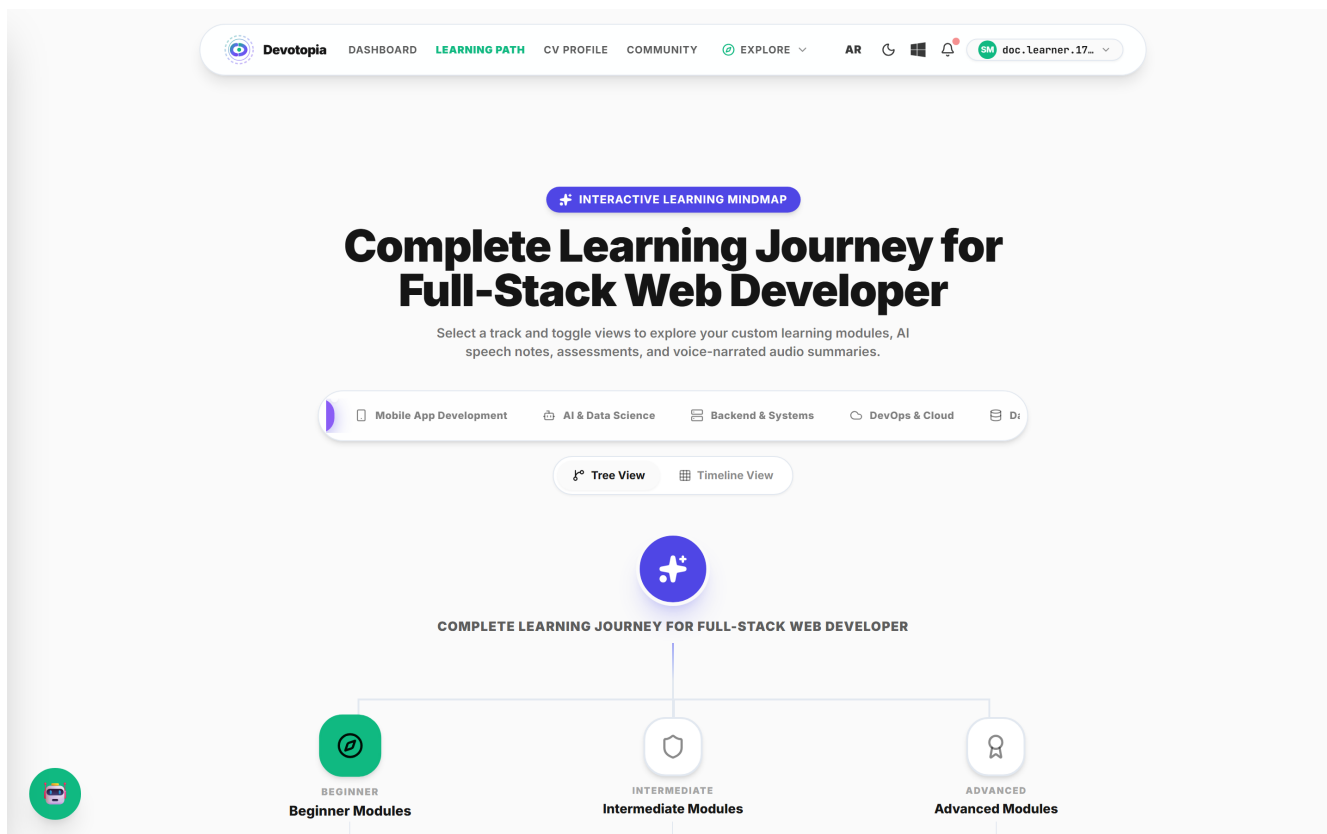


Figure 4.26 — The roadmap in Arabic. The tree mirrors with the text direction.

The interface is responsive down to mobile widths and installable as a progressive web application, which was the basis for excluding native applications in section 1.3.2.

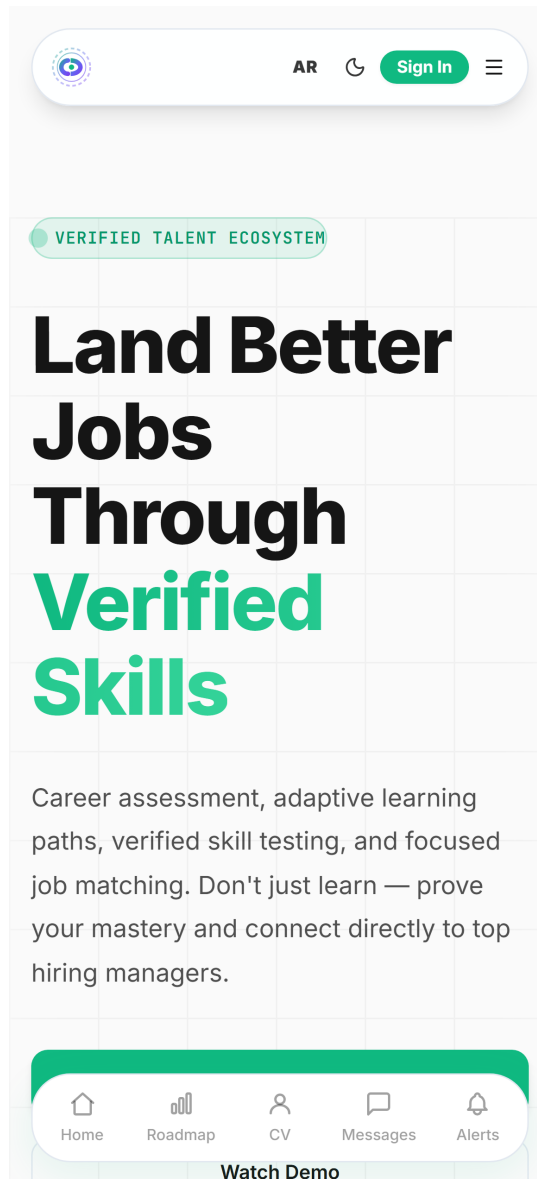


Figure 4.27 — Landing page at a 390-pixel mobile viewport.

4.3 Backend Technology

4.3.1 The need for a backend framework

An unstructured server would have been sufficient to serve the endpoints, but not to enforce the security properties in section 3.2.4. Three requirements in particular argued for a framework with first-class dependency injection and a request pipeline: a guard that must run before every controller without being attached by hand; validation that must occur at the boundary rather than inside handlers; and services that must be independently testable in isolation from HTTP.

4.3.2 Framework selection: NestJS

Framework	Strengths	Weaknesses	Decision
Express (unstructured)	Minimal; complete freedom; smallest dependency footprint.	Structure must be invented; global guards and validation attached manually, so a forgotten line becomes an unprotected route.	Rejected — the failure mode is silent and security-relevant.
NestJS	Modular dependency injection; global guards, interceptors and pipes; decorator-driven validation; OpenAPI generation; testable services.	Steeper initial learning curve; more ceremony for trivial endpoints.	Adopted — deny-by-default is a framework property here, not a convention.
Fastify (unstructured)	Higher raw throughput than Express.	Same structural weakness as Express; throughput was never the binding constraint.	Rejected.
Django / Laravel	Batteries included; mature admin tooling.	A second language in the stack, losing the shared type contract with the client.	Rejected — the shared contract was worth more.

4.3.3 Database selection: MongoDB

Option	Fit to this domain	Decision
Relational (PostgreSQL / MySQL)	Strong for fixed schemas and relational integrity. But a roadmap is a variable-length tree of modules with variable topic and prerequisite lists, and an assessment session holds a variable answer array — modelling these relationally means join tables whose rows are never queried independently.	Rejected for the core entities.
Document (MongoDB)	A roadmap with embedded modules, and a session with embedded answers, are each retrieved and written as a unit. Embedding matches the access pattern, and the flexible schema accommodates generated content whose shape varies.	Adopted.
Graph (Neo4j)	The prerequisite relationship is genuinely a graph.	Rejected — the graph is small and traversed only within one roadmap; a second database was not justified.

Embedding is the reason module and answer records have no independent collection. They are always read together with their parent, so storing them separately would add a query without adding capability.

4.4 Backend Development

4.4.1 Database model

The domain is persisted as 49 Mongoose schemas. The ten core entities and their relationships are shown below.

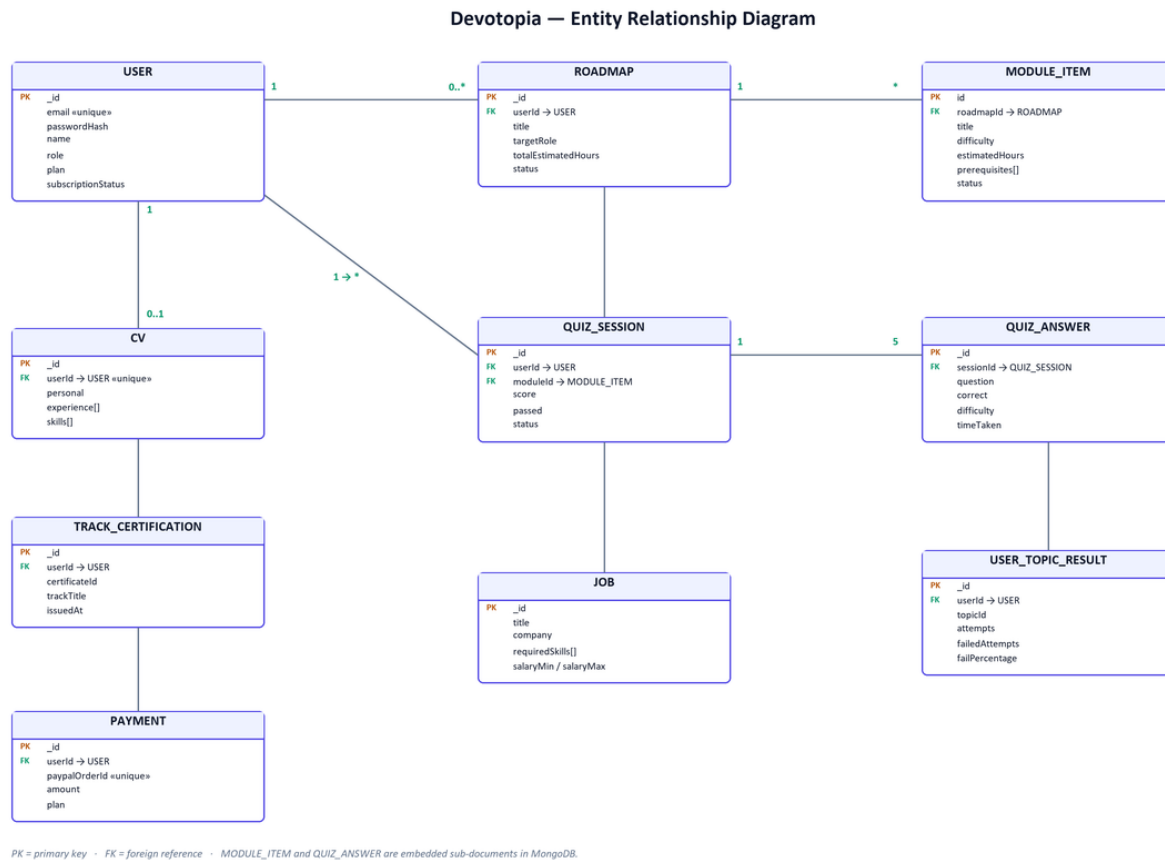
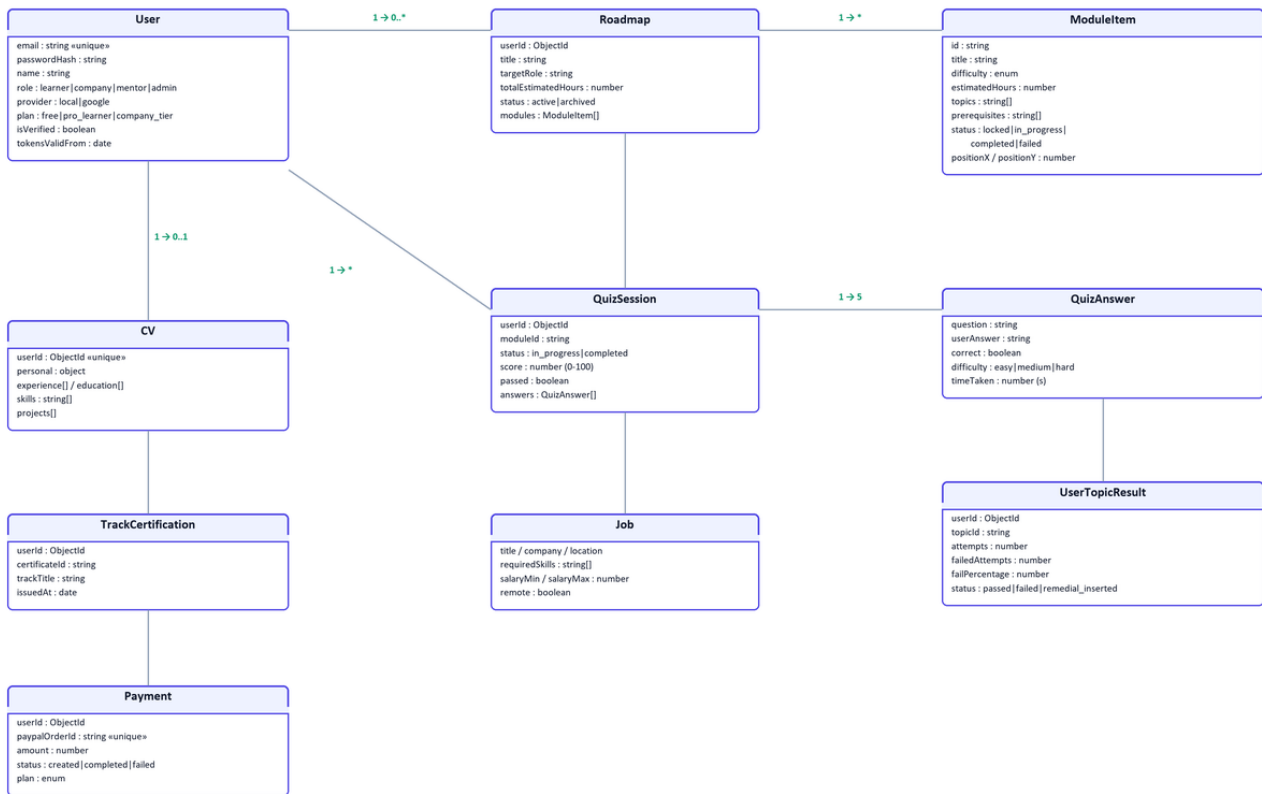


Figure 4.2 — Entity relationship diagram.

Devotopia — Class Diagram (core domain)



Cardinalities shown on each association. Sub-documents (ModuleItem, QuizAnswer) are embedded, not separate collections.

Figure 4.3 — Class diagram of the core domain.

Principal entities

Entity	Key attributes	Notes
User	email (unique), passwordHash, name, role, provider, plan, subscriptionStatus, tokensValidFrom	passwordHash is excluded from queries by default. tokensValidFrom invalidates every prior session when advanced.
Roadmap	userId, title, targetRole, totalEstimatedHours, status, modules[]	Modules are embedded, not referenced.
ModuleItem	id, title, description, difficulty, estimatedHours, topics[], prerequisites[], status, positionX, positionY	Prerequisites reference sibling module identifiers. Coordinates are produced by generation, so tree shape is data rather than a browser-side guess.
QuizSession	userId, moduleId, status, score, passed, answers[]	One session per attempt.
QuizAnswer	question, userAnswer, correct, difficulty, timeTaken	Each answer carries its own difficulty — this is what makes weighted scoring possible after the fact.
UserTopicResult	userId, trackId, topicId, attempts, failedAttempts, failPercentage, status	Drives the remedial trigger.
CV	userId (unique), personal, experience[], education[], skills[], projects[]	One per learner.
Job	title, company, location, requiredSkills[], salaryMin, salaryMax, remote	Matched against verified skills.
TrackCertification	userId, certificateId, trackTitle, issuedAt	Issued automatically on track completion.
Payment	userId, paypalOrderId (unique), amount, status, plan	The unique order identifier prevents double capture.

4.4.2 Security aspects

Security is enforced in layers, each of which is independent of the others. The design principle is that a route is protected unless it explicitly opts out, so that forgetting to protect a new endpoint is not possible.

Control	Mechanism	Threat addressed
Authentication	A guard registered globally, verifying the access token before any controller executes.	Unauthenticated access, including to routes added later without thought.
Authorisation by role	A separate roles guard evaluated per route.	A learner reaching employer or administrative functionality.
Ownership	An explicit assertion that the requesting user owns the resource, or is an administrator.	Insecure direct object reference — reading another learner's roadmap or session by changing an identifier.
Credential storage	bcrypt at 12 rounds, with the hash excluded from queries by default.	Disclosure of credentials if the database is exposed.
Session handling	Refresh token in an httpOnly cookie scoped to the authentication path; access token held in memory only.	Token theft by injected page script — a cookie the page cannot read cannot be exfiltrated by it.
Token rotation	Refresh tokens hashed per device, with reuse detected.	Replay of a stolen refresh token.
Mass invalidation	A validity timestamp on the user record.	Sessions surviving a password reset.
Privilege escalation	The administrator role rejected at the validation layer.	Self-promotion at registration.
Abuse resistance	Rate limiting, stricter on authentication routes.	Credential stuffing and enumeration.
Transport and headers	Helmet — content security policy, strict transport security, frame and referrer policy.	Cross-site scripting, clickjacking, protocol downgrade.
Payment integrity	Prices resolved server-side; webhook signatures verified before entitlement changes.	Tampered checkout amounts and forged payment notifications.

Two properties of this design are worth stating explicitly. First, user enumeration is prevented by returning an identical response for an unknown email and an incorrect password. Second, a production build refuses to start without payment credentials configured, so the mock payment path used in development cannot be reached in production.

These controls are verified by an executable suite of 44 checks run against a live server, covering authorisation boundaries, direct object references, identity spoofing, privilege escalation and rate limiting.

4.5 The Adaptive Assessment Engine

This section documents the central subsystem. It is presented in the order the logic executes: session creation, difficulty selection, scoring, outcome, and the remedial loop.

4.5.1 Overview

An assessment is five multiple-choice questions on one module. Three properties distinguish it from a conventional quiz: the difficulty of each question after the first pair responds to the learner's recent answers; the final score is weighted by the difficulty actually encountered; and failure is treated as an input to curriculum generation rather than as a terminal state.

4.5.2 Session creation

A session records the learner, the module, and an array of answers. Questions are generated on demand rather than drawn from a fixed bank, which is what allows the platform to assess a module whose subject was itself generated moments earlier. The first batch is requested at medium difficulty, since no evidence about the learner exists yet.

4.5.3 Difficulty selection

After each answer, the engine examines the two most recent answers to determine the difficulty of the next question.

Condition on the last two answers	Action	Resulting transition
Both correct	Raise one step	easy → medium → hard → hard
Both incorrect	Lower one step	hard → medium → easy → easy
One correct, one incorrect	Hold	unchanged

Why two answers and not one

This is the parameter most often questioned, and the reasoning is as follows. On a five-question assessment, adapting after a single answer means one lucky guess raises the difficulty and one careless slip lowers it. Either error consumes a substantial proportion of the remaining questions before it can correct itself. Requiring two consecutive answers in the same direction filters the majority of single-event noise while still allowing the ladder to traverse its full range within five questions.

Why the ladder saturates rather than wrapping

At the extremes the transition holds rather than wrapping — two correct answers at hard leave the learner at hard. The alternative would be to introduce further levels, which cannot be calibrated without the response data discussed in section 2.1.3.2.

4.5.4 Weighted scoring

The final score is not the proportion of questions answered correctly. Each answer carries the difficulty at which it was posed, and contributes according to that difficulty.

Difficulty	Weight
Easy	1.0
Medium	1.5
Hard	2.0

The score is the total weight earned divided by the total weight available, expressed as a percentage.

Why weighting is necessary

Without it the difficulty ladder is decorative. Consider two learners who each answer four of five questions correctly. The first remained on easy questions throughout; the second climbed to hard. Under unweighted scoring both record

80 per cent, and the record cannot distinguish them — which means the ladder has produced no information. Under weighted scoring the second learner scores materially higher, and the verified skill carries the difficulty at which it was demonstrated. That is precisely the property the employer-facing pipeline depends on.

Worked example

Question	Difficulty	Weight	Correct	Earned
1	Medium	1.5	Yes	1.5
2	Medium	1.5	Yes	1.5
3	Hard	2.0	No	0.0
4	Hard	2.0	Yes	2.0
5	Hard	2.0	No	0.0
Total	—	9.0	3 of 5	5.0

Three of five answers correct is 60 per cent unweighted. Weighted, the learner earns 5.0 of 9.0, which is 56 per cent — below the threshold. The learner reached hard questions, which the ladder correctly recorded, but did not demonstrate command of them.

4.5.5 Outcome and unlocking

The pass threshold is 70 per cent of the weighted score.

Why 70 per cent

The threshold has to sit above what guessing produces and below what perfection demands. With four options per question, random guessing yields an expected 25 per cent. A threshold at 50 per cent would certify learners who answered half correctly, which for a screening signal is too weak — an employer acting on it would frequently be misled, and the credibility of the whole record would degrade. Above roughly 80 per cent the failure rate rises to the point where the remedial loop dominates the learner experience. Seventy per cent sits between those failure modes.

On a pass, the module is marked complete, the prerequisite graph is walked to unlock every module whose prerequisites are now satisfied, a verified skill is recorded, and the system checks whether the whole track is complete and a certificate due. On a failure, the module is marked failed and control passes to the remedial mechanism.

4.5.6 The remedial loop

This is the mechanism that distinguishes the platform from those surveyed in chapter 2, and it is the answer to the failure mode described in section 1.2C.

- The incorrectly answered questions and their associated topics are collected from the session.
- The running statistics for that topic are updated: attempts, failed attempts, and the fail percentage derived from them.
- If the fail percentage reaches the remedial threshold of 30 per cent and this attempt was a failure, remedial generation is queued.
- The generation request carries the specific missed topics, so the resulting module addresses what the learner actually got wrong rather than the module subject in general.
- The generated module is shorter than a standard module and is inserted into the roadmap ahead of the failed one.

Why a running percentage rather than a single failure

Triggering on any single failure would generate a remedial module for a learner who was interrupted, guessed badly once, or simply had a poor day — which is noise, and which would make the roadmap grow without cause. A running percentage per topic distinguishes an isolated failure from a pattern. A learner who has attempted a topic four times and failed once sits at 25 per cent and is not interrupted; one who has failed two of four sits at 50 per cent and is.

Why generation is queued rather than synchronous

Generating a module takes seconds. Performing that work inside the request that submits the final answer would leave the learner waiting on a progress indicator at precisely the moment they have just been told they failed. The work is dispatched to a queue and the result appears in the roadmap shortly afterwards. Where the queue backend is unavailable, an in-memory implementation is used so that the absence of infrastructure degrades timing rather than function.

4.5.7 Known limitations

Two limitations are stated here rather than left to be discovered.

- **Sample size** — five questions is a small sample. In practice the ladder reaches hard only when a learner answers the opening pair correctly, so the top of the range is exercised less often than the design allows. A larger item bank with per-question statistics is the remedy, recorded in chapter 7.
- **Uncalibrated weights** — the weights of 1.0, 1.5 and 2.0 are reasoned rather than derived from response data. They produce a defensible ordering but not a psychometrically calibrated score, and scores are therefore comparable within a module rather than across the platform.

4.6 AI Curriculum Generation and Provider Orchestration

4.6.1 What is generated

Artefact	Input	Output
Roadmap	Target role, current skills, education, experience.	Structured modules with identifiers, titles, descriptions, difficulty, hour estimates, topics, prerequisite references and layout coordinates.
Assessment batch	Module title and topics, requested difficulty.	Five multiple-choice items with options, correct answer, difficulty label and explanation.
Remedial module	The specific topics missed in a failed attempt.	A shorter module scoped to those topics.
CV enhancement	An original experience statement.	A rewritten statement, with the original preserved.
Assistant reply	Learner question plus retrieved context.	A grounded answer.
Embeddings	Resource, FAQ and posting text.	Vectors for cosine-similarity retrieval.

4.6.2 The provider chain

Every model call passes through a single chain of providers, attempted in order and always terminating in a deterministic mock.

Order	Provider	Models used
1	OpenAI	gpt-4o-mini; text-embedding-3-small; tts-1
2	Google Gemini	gemini-2.5-flash; gemini-2.0-flash; text-embedding-004
3	Groq	llama-3.3-70b-versatile
4	Hugging Face	meta-llama/llama-3.1-8B-Instruct
5	Mock	Deterministic local generation — never fails

A provider that raises an error is skipped. Equally important, a provider that returns a syntactically valid response failing the expected shape check is also skipped — a reply without a populated module array is discarded and the next provider attempted. Treating a malformed response as equivalent to an outage is what prevents a plausible but unusable reply from reaching the learner.

4.6.3 Degradation observed in practice

During preparation of this documentation, all three configured live providers failed simultaneously: the OpenAI account returned a quota error, and both Gemini and Groq rejected their credentials. The chain fell through to the mock provider and the platform continued to operate — roadmaps were generated, assessments started and were answered, scoring and unlocking behaved normally. Only the textual quality of generated content degraded, because the mock produces structurally valid but generic material.

This is reported because it is the strongest available evidence for NFR-11. The degradation path is not a theoretical safeguard; it was exercised by a real simultaneous failure and the platform remained usable throughout.

4.6.4 The boundary between model and logic

The architectural constraint that shaped this system is stated here in full.

Decided by the model	Decided by deterministic code
Which modules constitute a curriculum for a role	Which module a learner may attempt next
What a module contains and how long it takes	Whether a module is locked, in progress, completed or failed
What questions are asked and at what stated difficulty	Which difficulty is requested next

Decided by the model	Decided by deterministic code
How a remedial module is written	Whether a remedial module is generated at all
How an experience statement is phrased	Whether an assessment is passed

The reason for the boundary is testability. Every rule in the right-hand column is a pure function of recorded state, so each can be unit-tested and each produces the same result on the same input. Placing any of them inside a model call would make a decision about a person non-deterministic, untestable, and dependent on the availability of an external service.

This is also the honest answer to whether the system is agentic: it is not. The sequence is fixed by application code, and the model never chooses which operation to perform next. That was a deliberate choice, and section 4.5 explains what it buys.

4.6.5 Retrieval

Learning resources, frequently asked questions and job descriptions are embedded and stored in a Qdrant collection using cosine similarity. Retrieval grounds assistant replies and resource recommendations in real material rather than model recall. Where the vector store is unavailable the retrieval layer returns curated fallbacks, so the feature degrades rather than failing.

The present implementation uses fixed-size chunking, which is known to truncate context at chunk boundaries. Section 7.2 records the intended replacement.

4.7 External Services Integration

Service	Purpose	Failure behaviour
PayPal	Subscription orders, capture and webhooks.	Payment fails visibly; entitlement is unchanged; a production build will not start without credentials.
Cloudinary	Image and media storage with delivery.	Upload fails; existing media unaffected.
Appwrite	Object storage for generated audio.	Audio features unavailable; the platform is otherwise unaffected.
Resend	Transactional email — verification and password reset.	Messages are logged to the server console instead, so development proceeds without credentials.
AssemblyAI	Speech transcription for voice features.	Voice input unavailable; text input unaffected.
Adzuna	Salary insight data.	The insight panel is omitted.
Piston	Sandboxed execution of submitted code.	Falls back to a mock run rather than executing untrusted code in-process.
Qdrant	Vector retrieval.	Curated fallbacks are returned.
Redis	Queue backend for remedial generation.	An in-memory queue is used.

Every entry in the final column is a deliberate decision. The pattern throughout is that the failure of an auxiliary service removes a feature but never blocks the core loop of learning, assessment and verification.

4.8 Performance Engineering

Performance was measured before it was optimised, using a Playwright project that records navigation timings, web vitals, transferred bytes and main-thread task duration against a production build.

4.8.1 Measurement method

Measuring against the development server was rejected as misleading: it ships unminified code and compiles routes on demand, producing figures no user would experience. All figures below were taken from a production build served locally, with the performance project running single-worker so that parallel pages do not compete for processor time.

4.8.2 Findings and remedies

Finding	Cause	Remedy	Result
Two heaviest routes slow to become interactive.	A 309 kB charting bundle in the initial payload, though no chart is needed to paint the page.	Charts extracted to one module and dynamically imported after hydration.	Dashboard 316 kB → 161 kB; quiz 243 kB → 131 kB.
Assessment route carried a full charting library.	The countdown ring was a radial chart.	Replaced with a stroke-dashoffset SVG circle.	Charting removed from the route entirely.
Nine routes carried a 121 kB animation library.	The full library imported for simple entrance effects.	On-demand loading of the DOM animation feature set only.	Approximately 24 kB removed from each affected route.
Sign-up pages: fast first paint but slow largest paint.	The largest element was animated in after a deliberate 0.4 s delay with zero opacity.	Entrance tightened and the opacity fade removed so the element paints immediately.	LCP 1,540 ms → 272 ms.

4.8.3 Measured outcome

Route	TTFB	FCP	LCP	CLS	Longest task
/	8 ms	264 ms	264 ms	0	0 ms
/pricing	9 ms	236 ms	236 ms	0	0 ms
/about	6 ms	196 ms	196 ms	0	0 ms
/auth/login	6 ms	208 ms	208 ms	0	0 ms
/auth/register	4 ms	208 ms	272 ms	0	0 ms

Table 4.1 — Page timings on a production build, single worker.

For reference, a largest contentful paint below 2.5 seconds and a cumulative layout shift below 0.1 are the thresholds commonly treated as good. Every route measured is an order of magnitude inside the first and at zero on the second.

4.9 Testing and Quality Assurance

Layer	Tool	Coverage
Unit	Jest	Domain rules, notably the remedial trigger and its threshold behaviour.
Integration	Supertest	HTTP behaviour of controllers within the application context.
End-to-end	Playwright	Stack boot, registration through the real two-step form, community and administrative authorisation.
Security	Custom suite	44 checks against a live server: authorisation, direct object references, spoofing, escalation, rate limiting.
Performance	Playwright	Navigation timings and web vitals with explicit budgets.
Static	TypeScript, ESLint, Prettier	Type safety across both applications through the shared schema package.

Two testing decisions are worth recording. First, the end-to-end suite runs single-worker, because the authentication rate limiter is deliberately strict and parallel workers registering accounts would trip it — causing the suite to fail on throttling rather than on behaviour. Second, the registration test drives the real two-step form rather than calling the API directly, because the page reports every failure identically and only a browser-level test distinguishes a genuine network fault from a validation refusal.

5. Deployment

5.1 Deployment options

Option	Characteristics	Assessment
Platform as a service (Vercel, Railway, Render)	Managed builds and deployment from the repository; automatic certificates; minimal operational work.	Well suited to the client. The server's long-running queue worker fits less comfortably in function-oriented hosting.
Containers on a virtual machine	Both applications as images behind a reverse proxy.	Adopted as the reference deployment. Dockerfiles exist for both applications and reproduce the local environment exactly.
Managed Kubernetes	Full orchestration and horizontal scaling.	Rejected — operational overhead disproportionate to present scale.

5.2 Reference deployment

Component	Deployment	Requirement
Web client	Container from the Next.js standalone output.	1 vCPU, 512 MB.
API server	Container.	1 vCPU, 1 GB minimum; 2 vCPU, 2 GB recommended.
MongoDB	Atlas, managed.	M0 for evaluation; M10 or larger for production traffic.
Qdrant	Container, optional.	1 vCPU, 1 GB.
Redis	Container, optional.	256 MB.
Code sandbox	Piston container, optional.	Isolated from the application network.

5.3 Configuration

Configuration is supplied by environment variables and validated on startup, so a misconfigured deployment fails immediately and visibly rather than at the first request that needs the missing value.

Variable	Required	Note
MONGODB_URI	Yes	No silent fallback to localhost; the server will not start without it.
JWT_SECRET, JWT_REFRESH_SECRET	Yes	At least 32 characters each. Changing either invalidates all existing sessions.
PORT	Yes	Must match the API URL the client is built with.
FRONTEND_URL	Yes	An explicit cross-origin allow-list, never a wildcard.
PAYPAL_CLIENT_ID, PAYPAL_CLIENT_SECRET	In production	A production build refuses to start without these, so mock payments cannot reach production.
Model provider keys	No	Absent keys cause the chain to fall to the mock; the platform remains usable.
QDRANT_URL, REDIS_HOST	No	Absent values select the documented fallbacks.

5.4 Operational notes

Three configuration faults were encountered during development that present very differently from their causes, and are recorded here because each cost significant time to diagnose.

- **Port mismatch** — the client is built against a fixed API URL. If the server's port does not match it, every authenticated request fails with a generic network error while both processes appear to be running correctly. The symptom points at the network; the cause is configuration.
- **Unset realtime endpoint** — if the socket endpoint is not configured the client falls back to a default port and retries indefinitely, producing a continuous stream of connection errors unrelated to any user action.
- **Startup-critical optional services** — two services read configuration in a manner that aborts startup when the value is absent, even though the features they serve are optional. Placeholder values are sufficient for development.

6. Conclusion

This project set out to address a single problem observed from two directions: the absence of a trustworthy, portable signal of what a junior developer can actually do. The learner cannot demonstrate ability, and the employer cannot detect it.

The platform delivered addresses that problem by making learning and verification the same activity. A curriculum is generated for a stated target role and ordered by prerequisite. Each module is gated by an assessment whose difficulty responds to the learner and whose score reflects the difficulty they faced. Passing produces a verified skill carrying that difficulty; failing produces a shorter module built from what the learner actually got wrong. The resulting record is exposed to employers as a ranked pipeline, and gaps found against real postings can be returned to the learner's roadmap in a single action.

6.1 What was achieved

Objective	Outcome
Generate a personalised, prerequisite-ordered curriculum	Achieved — produced in a single request from role, skills and experience.
Adapt assessment difficulty to the learner	Achieved — two-answer streak transitions, verified by unit test.
Score by demonstrated difficulty rather than question count	Achieved — weights of 1.0, 1.5 and 2.0 applied per answer.
Respond to failure by changing what is studied	Achieved — remedial generation on a 30 per cent running fail rate.
Expose verified skill to employers	Achieved — pipeline ranked by exam performance and completion.
Remain usable when model providers fail	Achieved — demonstrated under real simultaneous provider failure.
Serve Arabic and English equally	Achieved — full translation with layout mirroring.
Meet performance and security targets	Achieved — LCP under 300 ms, CLS 0, 44 security checks passing.

6.2 What was learned

- **Constraining the model was the decision that mattered** — the instinct on a project of this kind is to apply the model as widely as possible. Restricting it to content generation, and keeping every outcome-determining branch in deterministic code, is what made the assessment rules testable — and it is why the platform survived a total provider outage. The constraint produced more capability than its removal would have.
- **Measure before optimising** — the initial performance figures taken against the development server suggested payloads of two megabytes. Against a production build the real figure was a fraction of that, and the actual problem was a different one — a decorative animation delay costing more than a second of largest contentful paint. Optimising the first set of numbers would have consumed effort on a problem that did not exist.
- **A parameter is only defensible if its reasoning is recorded** — the thresholds in the assessment engine — two answers, 70 per cent, 30 per cent — are each defensible, but only because the reasoning was written down at the time. A number without its justification cannot be defended later, and cannot be revised safely either.
- **Degradation paths must be exercised, not assumed** — the provider fallback chain was written early and believed to work. It was only trusted after it was observed handling a real simultaneous failure of every configured provider.

6.3 Limitations

The following are stated plainly rather than left for a reader to find.

- **Assessment resolution** — five questions is a small sample; the difficulty ladder is exercised less fully than the design permits, and weights are reasoned rather than calibrated.
- **Job data** — postings are seeded and employer-supplied; automated ingestion is not implemented.
- **Retrieval quality** — fixed-size chunking truncates context at boundaries, which limits assistant answer quality on longer material.
- **Scale** — the system is designed to scale horizontally but has not been load-tested; the scalability requirement is deferred rather than demonstrated.

7. Future Plans

7.1 Assessment quality

- **Larger item bank with response statistics** — accumulate per-question difficulty and discrimination data from real attempts, which is the prerequisite for everything below.
- **Item response theory** — with calibration data available, replace the fixed weights with an IRT model so that scores become comparable across modules rather than only within one. This was rejected initially for the reason given in section 2.1.3.2, and the obstacle is data rather than design.
- **Assessment integrity** — question randomisation, response-time analysis to detect answer sharing, and optional proctoring for credentials intended to carry more weight.
- **Free-text and code assessment** — graded by a deterministic test harness rather than by model judgement, preserving the boundary set out in section 4.6.4.

7.2 Retrieval and model usage

- **Sentence-window retrieval** — index at sentence granularity and expand to neighbouring sentences after matching, giving precise retrieval without truncated context.
- **Auto-merging retrieval** — index a hierarchy of parent and child chunks; when enough children of one parent match, substitute the parent so multi-paragraph material is restored intact.
- **Retrieval evaluation** — adopt context relevance, groundedness and answer relevance as measured metrics, so retrieval quality becomes an observed quantity rather than an impression.
- **Cost control** — per-user token budgets and caching of repeated generations, to keep inference cost predictable as usage grows.

7.3 Product

- **Employer-verified outcomes** — record whether a matched candidate was interviewed or hired, closing the feedback loop on match quality and providing the data to improve ranking.
- **Cohort accounts** — dashboards for bootcamps and training centres showing per-skill outcomes across a group.
- **Live job ingestion** — a compliant feed so that postings remain current without manual seeding.
- **Native applications** — beyond the current progressive web application, if usage justifies the additional codebases.

7.4 Engineering

- **Performance budgets in continuous integration** — fail the build when a change pushes largest contentful paint or bundle size past the agreed threshold, so the improvements in section 4.8 are not lost over time.
- **Broader automated coverage** — extend the end-to-end suite across the full learner journey, including payment flows against the sandbox.
- **Observability** — structured logging, error tracking and per-endpoint latency in production.
- **Secret management** — move every credential to a managed secret store, and rotate anything that has previously appeared in the repository.
- **Load testing** — establish the actual capacity of the reference deployment, converting the deferred scalability requirement into a measured one.

8. Citations and References

References are listed by category. Entries marked with a placeholder require the accession date to be completed before submission.

8.1 Frameworks and libraries

- Next.js Documentation. Vercel Inc. <https://nextjs.org/docs>
- React Documentation. Meta Open Source. <https://react.dev>
- NestJS Documentation. <https://docs.nestjs.com>
- Mongoose Documentation. MongoDB Inc. <https://mongoosejs.com/docs>
- MongoDB Manual. MongoDB Inc. <https://www.mongodb.com/docs/manual>
- Qdrant Documentation. Qdrant. <https://qdrant.tech/documentation>
- Tailwind CSS Documentation. <https://tailwindcss.com/docs>
- Playwright Documentation. Microsoft. <https://playwright.dev/docs/intro>
- BullMQ Documentation. <https://docs.bullmq.io>
- Zod Documentation. <https://zod.dev>

8.2 Model providers

- OpenAI API Reference. OpenAI. <https://platform.openai.com/docs>
- Gemini API Documentation. Google. <https://ai.google.dev/docs>
- Groq API Documentation. Groq Inc. <https://console.groq.com/docs>
- Hugging Face Inference Documentation. <https://huggingface.co/docs>
- AssemblyAI Documentation. <https://www.assemblyai.com/docs>

8.3 Methods and standards

- Kano, N., Seraku, N., Takahashi, F. and Tsuji, S. "Attractive Quality and Must-Be Quality." Journal of the Japanese Society for Quality Control, 1984.
- Sommerville, I. Software Engineering. Pearson. — Requirements engineering process referenced in chapter 3.
- Lord, F. M. Applications of Item Response Theory to Practical Testing Problems. Routledge. — Basis of the approach discussed in sections 2.1.3.2 and 7.1.
- Web Vitals. Google. <https://web.dev/vitals> — Definitions and thresholds used in sections 3.2.4 and 4.8.
- OWASP Top Ten. OWASP Foundation. <https://owasp.org/www-project-top-ten> — Threat categories addressed in section 4.4.2.
- PayPal REST API Reference. PayPal. <https://developer.paypal.com/api/rest>

8.4 Platforms examined

- Coursera. <https://www.coursera.org> — Examined for the comparison in section 2.1.1.1.
- LinkedIn Learning and Skill Assessments. <https://www.linkedin.com/learning>
- roadmap.sh. <https://roadmap.sh> — Community learning roadmaps.
- Pluralsight Skill IQ. <https://www.pluralsight.com/product/skill-iq>

- Duolingo. <https://www.duolingo.com> — Difficulty adaptation and streak mechanics.
- Khan Academy. <https://www.khanacademy.org> — Mastery-based progression.

Before submission

Add the accession date to each web reference, and convert the list to the citation style your department requires.

Add citations for any statistics introduced into section 1.1 or section 2.3, with source and year — those are the entries assessors are most likely to check.